nanuq

MANUAL TÉCNICO

Cómo está hecho Nanuq

Arquitectura con diagramas, cada plataforma por dentro y, al final, cada función del código.





Versión 1.48.0 · 12 de noviembre de 2026 · Español · Software libre (GPL-3.0-or-later)



Índice

La parte 1 explica el diseño; la parte 2 es la referencia de cada función.

ARQUITECTURA

1 **Panorama** 6
Una sola interfaz, tres envolturas.

2 **El repositorio** 8
Qué hay en cada carpeta.

3 **La página** 9
Cómo arranca, navega y reacciona app.js.

4 **Datos y almacenamiento** 11
Lo que dice algo de ti va cifrado.

5 **Reproducción** 13
De dónde sale cada canción y qué pasa cuando algo falla.

6 **Biblioteca y Jellyfin** 15
La API que usa Nanuq y lo que arma por su cuenta.

7 **DJ Nanuq** 16
Rondas con reglas, sin IA.

8 **Mis dispositivos** 17
Un Nanuq suena y los demás lo controlan, a través de Jellyfin.

9 **La fiesta** 18
Teléfonos que mandan canciones a una computadora por la red local.

10 **La app de Android** 20
WebView + Java nativo.

11 **La app de escritorio** 21
Electron para Linux y Windows.

12 **Seguridad** 22
Lo que llega de afuera nunca se cree.

13 **Compilar y publicar** 23
Una orden para todas las plataformas.

14 **Convenciones y trampas** 24
Lo que conviene saber antes de cambiar algo.

15 **Licencia** 25
GPLv3 o posterior, archivo por archivo.

NIVEL PROFUNDO: CADA FUNCIÓN

16 **web/app.js** 28
604 funciones, en el orden del archivo.

Base: ayudantes y almacén 28
Direcciones del servidor 29
API de Jellyfin 29
Audio: formas de onda y sintetizador 31
Biblioteca demo 31
Ajustes y temas 33
Descargas (Android) 33
Estado y navegación 34
Álbumes según las etiquetas 34
Búsqueda en las letras 35
Ayudantes de dibujo 35
Vistas 36
DJ Nanuq 36
Tu año y Tu semana (video) 39
Solicitar canción (SomeDL) 40
Reproductor 41
Letras 43
Cola 44
Avisos, menús y diálogos 44
Mis dispositivos 45
Fiesta (watch party) 47
Acciones globales 49
Entrar 49

17 **Resto de la página** 50
Idiomas y service worker.

web/i18n.js 50

web/sw.js 50

18 **Android** 51
Java nativo y el puente de JavaScript.

android/app/src/main/java/com/nanuq/app/Discovery.java 51
android/app/src/main/java/com/nanuq/app/Downloads.java 51
android/app/src/main/java/com/nanuq/app/LocalServer.java 52
android/app/src/main/java/com/nanuq/app/Page.java 53
android/app/src/main/java/com/nanuq/app/MainActivity.java 54
android/app/src/main/java/com/nanuq/app/MediaCenter.java 54
android/app/src/main/java/com/nanuq/app/MediaButtonReceiver.java 56
android/app/src/main/java/com/nanuq/app/Resume.java 56
android/app/src/main/java/com/nanuq/app/NanuqBrowserService.java 56
android/app/src/main/java/com/nanuq/app/NativeHttp.java 56
android/app/src/main/java/com/nanuq/app/PlaybackService.java 56
android/app/src/main/java/com/nanuq/app/Prefetch.java 56
android/app/src/main/java/com/nanuq/app/SecureStore.java 57
android/bridge/android-bridge.js 58
android/build-apk.sh 58
android/app/src/main/java/com/nanuq/app/OggCheck.java 58
android/app/src/main/java/com/nanuq/app/NativeAudio.java 58

19 **Escritorio** 60
Electron y su empaquetado.

linux/main.js 60
linux/preload.js 62
linux/prepare-web.js 62
linux/collect.js 62
linux/flatpak/build-flatpak.sh 62

20 **Compilación** 63
El script que compila todo.

scripts/build.sh 63

PARTE 1

Arquitectura

Cómo está armado Nanuq: una sola interfaz web, las apps que la envuelven y cómo se habla con Jellyfin, con tus otros dispositivos y con la fiesta.

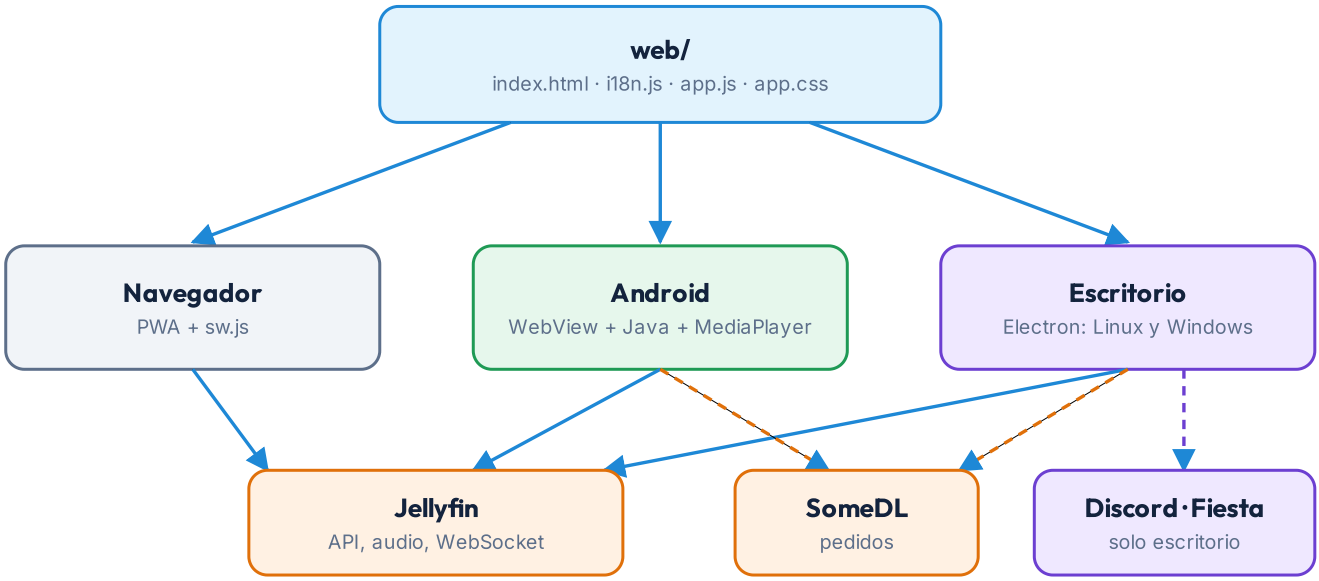


01 Panorama

Una sola interfaz, tres envolturas.



Toda la interfaz de Nanuq vive en `web/`: cuatro archivos sin compilar y sin dependencias. Las apps de Android y de escritorio no tienen interfaz propia: copian esa misma carpeta al compilar y le agregan, por un «puente», lo que una página web no puede hacer sola.



Cada plataforma sirve la misma página; las flechas punteadas son lo que solo hace el código nativo (SomeDL sin CORS, Discord, la sala de la fiesta).

Por qué así

- **Un cambio llega a todas partes**: la interfaz se toca solo en `web/`; Android y Linux la copian al compilar (`syncWeb` en Gradle, `prepare-web.js` en Electron).
- **Contexto seguro sin HTTPS**: Android sirve la página en `http://localhost` y Electron en `http://127.0.0.1:47863`. Así hay `crypto.randomUUID` y almacenamiento (el service worker solo se usa en el navegador), y a la vez se puede hablar con un Jellyfin `http://` de la red local sin bloqueos de contenido mixto.
- **Sin servicios de terceros**: Nanuq solo habla con tu Jellyfin, tu SomeDL y (si lo activas) la app de Discord de tu computadora. Las únicas peticiones externas son las fuentes de Google Fonts.

Puertos

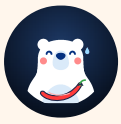
Puerto	Quién	Para qué
TCP 47863	Escritorio (127.0.0.1)	Sirve la página y, con la fiesta abierta, el audio de la sala (`/fiesta/<token>/…`). Puerto fijo porque el almacenamiento depende del origen.
TCP 47864	Escritorio (red local)	WebSocket de la fiesta (`/fiesta`) y carátulas para los invitados. Solo con la sala abierta y solo direcciones privadas.
UDP 47865	Escritorio	Responde a «Who is NanuqParty?» para que los teléfonos encuentren la sala (≤ 30 respuestas/s).
UDP 7359	Jellyfin	Búsqueda oficial de servidores («Who is JellyfinServer?»), desde Android y escritorio.
TCP aleatorio	Android (127.0.0.1)	`LocalServer` : descargas, carátulas y lo guardado por adelantado, con token en la ruta.
8096 / 8920	Jellyfin	Puertos que se asumen si escribes una IP o un nombre corto sin puerto.
5000	SomeDL	Puerto que se asume para SomeDL sin puerto.

02 El repositorio

Qué hay en cada carpeta.



Ruta	Contenido
`web/index.html`	Estructura, CSP estricta y `<defs>` SVG: íconos `#i-*` y el oso vectorizado (`#djb-*`).
`web/i18n.js`	Idiomas: `LANGS` , `LANG` , `LOC` , el diccionario `EN` (clave = texto en español) y `tx()` . Se carga antes que app.js.
`web/app.js`	Toda la lógica (6128 líneas). `const VERSION` es la fuente de verdad de la versión.
`web/app.css`	Todo el estilo, con variables por tema (18 temas).
`web/sw.js`	Service worker, solo en el navegador: guarda la app (red primero para el código), nunca música ni API. Android y escritorio no lo registran.
`android/`	WebView + Java (`com.nanuq.app`), el puente `bridge/android-bridge.js` , `build-apk.sh` y sus pruebas (`test/` : OggCheck y el reproductor del puente).
`linux/`	Electron: `main.js` , `preload.js` , `prepare-web.js` , `collect.js` , `flatpak/` y `test/` (pruebas de `main.js`). También arma el instalador de Windows.
`scripts/build.sh`	Sube la versión y compila todo a `dist/<versión>/` .
`docs/`	Imágenes del README y estos manuales (`docs/manuals/src` : `node docs/manuals/src/build.js`).



Ojo

`dist/` , `keystore.properties` y los `*.jks` no van al repositorio. La llave de firma del APK vive fuera (`~/.android/nanuq-release.jks`): sin ella no se pueden publicar actualizaciones.

03

La página

Cómo arranca, navega y reacciona app.js.



Arranque

1. Al cargar, `i18n.js` decide el idioma y `txStatic()` traduce los textos fijos del HTML.
2. `SEC` detecta el almacén cifrado nativo y `secMigrate()` mueve allí lo que versiones viejas dejaron en claro.
3. El bloque de arranque dibuja la barra lateral y las pestañas, aplica el tema e inicia las descargas (`DL.init`).
4. `Vault.init()` reintenta unos segundos si el Keystore o el llavero aún no responden; luego `Vault.load()` lee la sesión.
5. Con sesión: `startApp(new Jellyfin(sess))` recupera la cola y la posición (`QPOS`), restaura el DJ y abre Inicio. Sin sesión: `showLogin()`.

Navegación

`S.stack` es la pila de pantallas. `go(nombre, params, opts)` empuja una pantalla (las de `TOP` vacían la pila) y llama a `render()`, que ejecuta `V[nombre](caja, params)`. Cada vista es una función asíncrona que llena su caja; si deja `_cleanup`, se llama al salir. `back()` cierra primero lo que esté encima (menú, editor de letra, reproductor) y luego retrocede. Con la fiesta abierta (`party-host` / `in-party`) solo existe la pantalla de la fiesta.

Eventos

La CSP prohíbe scripts en línea, así que no hay `onclick=`: todo botón lleva `data-act="…"` y un único manejador de clic global decide qué hacer (sección «Acciones globales»). Las imágenes que deben desaparecer si fallan llevan `data-rm`.

Idiomas

Todo texto visible se escribe en español dentro de `tx('…')`; `i18n.js` tiene la traducción con el texto en español como clave. Con valores: `tx('… {0} …', [x])`. Cambiar de idioma guarda la posición y recarga la página, así también se traducen los textos que se arman al cargar.

Secciones de app.js

Sección	Desde la línea
Base: ayudantes y almacén	1

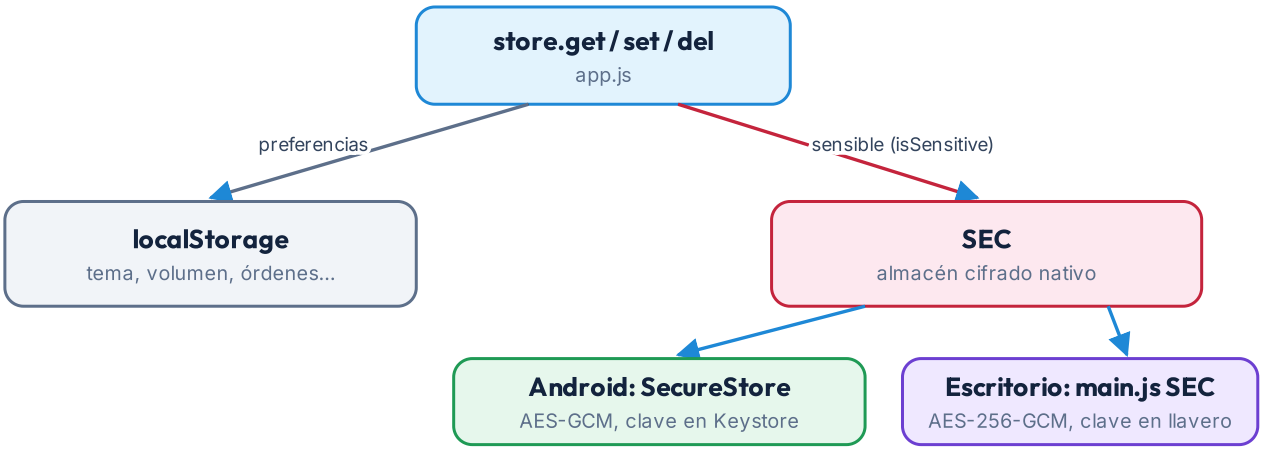
Sección	Desde la línea
Direcciones del servidor	122
API de Jellyfin	177
Audio: formas de onda y sintetizador	489
Biblioteca demo	550
Ajustes y temas	748
Descargas (Android)	808
Estado y navegación	967
Álbumes según las etiquetas	1017
Búsqueda en las letras	1122
Ayudantes de dibujo	1220
Vistas	1250
DJ Nanuq	1317
Tu año y Tu semana (video)	2277
Solicitar canción (SomeDL)	2988
Reproductor	3279
Letras	3863
Cola	4229
Avisos, menús y diálogos	4303
Mis dispositivos	4314
Fiesta (watch party)	5012
Acciones globales	5753
Entrar	5914
Arranque	6093

04 Datos y almacenamiento

Lo que dice algo de ti va cifrado.



`store` es el único acceso al almacenamiento. Las preferencias de la interfaz (tema, volumen, órdenes) van a `localStorage` con el prefijo `nanuq.`; lo que dice algo de ti va al almacén cifrado de la plataforma. Como las preferencias no van cifradas, se validan al leerlas (justo después de `Settings`): un valor alterado o de otra versión vuelve al de siempre.



En el navegador no hay almacén seguro: todo queda en localStorage (Ajustes lo dice).

Clave	Qué guarda
session	Servidor, usuario y token de Jellyfin.
queue · queuePos	Tu cola (hasta 3000) y el segundo donde ibas (queuePos no es sensible: solo un id al azar y segundos).
libIdx · lyrIdx	Índices de tu biblioteca (álbumes por etiquetas) y de tus letras. Cada uno en su propio archivo cifrado (`datos—<clave>.bin` , misma clave): si fueran con lo demás, cada cambio de canción volvería a cifrar y escribir varios MB.
dj · djHist	Sesión del DJ en curso e historial de lo que puso.
plays · mp3ids	Reproducciones por canción en este dispositivo y canciones que van en MP3.
somedl · somedlReqs · somedlLast	Servidor SomeDL y tus pedidos.
servers · lastServer · device · party	Servidores recientes, id del dispositivo y la fiesta en curso (teléfono).
lrc.<id>	Letras guardadas solo en este dispositivo.

Clave	Qué guarda
native.resume	Android (lo escribe Java, no la página): la última canción para la tarjeta de «reanudar» (`Resume`); su carátula va en el almacenamiento privado. Se borra al cerrar sesión.

Estados del almacén

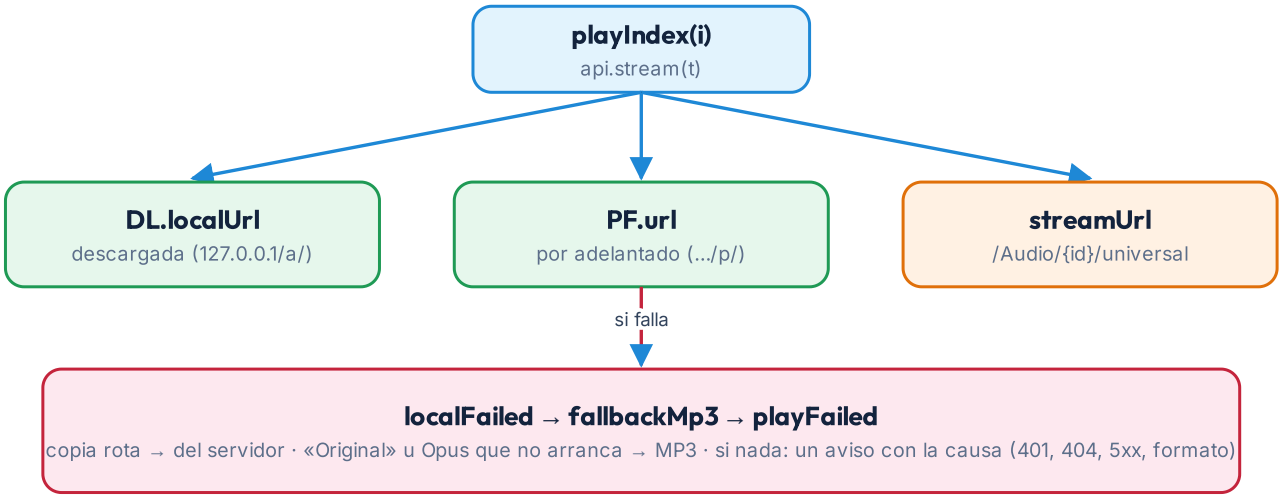
`SEC.kind` puede ser el nombre del almacén (funciona), `locked` (hay datos cifrados que ahora no abren: se reintenta), `lost` (la clave ya no existe: se ofrece «Empezar de cero», que aparta los datos sin borrarlos) o `null` (no hay almacén). Mientras no esté abierto, lo nuevo vive solo en memoria (`SEC.mem`): **nunca** se escribe en claro.

05 Reproducción

De dónde sale cada canción y qué pasa cuando algo falla.



`P` guarda la cola, el índice y dos reproductores (`P.els`, creados con `newAudio()`; `P.audio` es siempre el de la canción actual). En el navegador y el escritorio son `<audio>`; en Android, `NanuqNativeAudio`, que se usa igual pero suena con el reproductor del sistema. `playIndex(i)` elige la fuente con `api.stream()`: primero la copia descargada, luego la guardada por adelantado y, si no hay, el streaming de Jellyfin.



En Android, las dos primeras fuentes las sirve el teléfono mismo; en Android y escritorio la página nunca baja el audio completo a memoria.

Detalles que importan

- **Saltar en conversiones en vivo**: si Jellyfin transcodifica, el archivo no admite saltos; `seekTo` pide el audio otra vez con `StartTimeTicks` y recuerda el desplazamiento (`P.off`). Al arrastrar, la petición sale al soltar.
- **Atascos**: `armStallWatch` avisa si no arranca en 10 s; `stallCheck` reanuda desde el mismo segundo si la red se cayó a media canción; `netBack` lo hace al volver la red. Un error de red con la canción ya sonando no pasa al MP3 de respaldo: solo se espera a la red.
- **Calidad**: 96 kbps por omisión; `qualPaint` la muestra en el reproductor maximizado («Opus · 96 kbps», «Original», «En el teléfono»).
- **Formatos**: `streamUrl` deja sonar tal cual FLAC, Opus, AAC, MP3, WAV y OGG (en «Original» o bajo el bitrate); si hay que convertir, pide Opus en Ogg (`Settings.codec`, si el reproductor lo abre: `OPUS_OK`) o MP3. Lo que falló una vez va en MP3 (`MP3.ids`). Las descargas usan el mismo formato.
- **Desvanecer**: `pauseSoft` / `playSoft` con `ramp()`, lineal en decibelios (−50 dB = silencio), de 0.5 a 5 s (`Settings.fadeMs`). Mientras baja ya cuenta como pausa (`FADE.out`) y ▶ lo revierte. El volumen de

vuelta es siempre el tuyo (`userVol`). Al arrastrar la barra, rampas de 30–40 ms solo contra chasquidos (`P.scrub`).

- **Transición del DJ**: `xfCheck` en `timeupdate`; a 3–5 s del final (`Settings.xfade`), `xfStart` intercambia los dos `<audio>` y cruza con curvas de potencia constante (seno/coseno). Los eventos del que se va se ignoran (`onAudio`); `xfKill` lo apaga al pausar o cambiar de canción.
- **Pausa ajena**: si otra app tomó la salida de audio (`P.extPause`), al reanudar se recarga la canción para no quedarse mudo (no aplica al reproductor de Android).
- **Pantalla apagada** (`pageHidden`, solo navegador y Android): sin desvanecidos ni transición del DJ, porque sus temporizadores van frenados; al ocultarse a media rampa, `rampsFinish` la termina. El escritorio no se frena con la ventana minimizada.
- **Controles del sistema**: `navigator.mediaSession`; en Android lo reemplaza el puente nativo. En escritorio un audio en silencio (`rcKeep` / `sessionHold`) mantiene los controles mientras carga o controlas otro dispositivo.
- **Favoritas**: Jellyfin 12 sobrescribe la favorita de la canción que suena con cada aviso de reproducción; `FAVHOLD` la vuelve a guardar después de cada aviso.

06

Biblioteca y Jellyfin

La API que usa Nanuq y lo que arma por su cuenta.



Ruta de Jellyfin	Para qué
/System/Info/Public · /Users/AuthenticateByName	Reconocer el servidor y entrar.
/Items · /Artists/AlbumArtists · /MusicGenres · /Playlists	Listas de la biblioteca (con varias rutas de respaldo según la versión).
/Audio/{id}/universal	El audio (streaming, descargas, por adelantado, fiesta).
/Audio/{id}/Lyrics · .../RemoteSearch/Lyrics	Leer, subir y buscar letras (10.9+).
/UserFavoriteItems/{id}	Favoritas (y la ruta anterior a 10.9 como respaldo).
/Items/{id}/InstantMix	Radio de una canción.
/Sessions · /Sessions/{id}/Playing · /socket	Mis dispositivos: sesiones, órdenes y avisos en vivo.
/Sessions/Playing(/Progress|/Stopped)	Avisos de reproducción (lo que ve el servidor).
/DisplayPreferences/nanuq-dj	Estado del DJ compartido entre tus dispositivos.
/user_usage_stats/...	Plugin Playback Reporting para Tu semana / Tu año (si está).

Álbumes por etiquetas (LIB)

Jellyfin arma los álbumes por carpeta; Nanuq los arma con las etiquetas «Álbum» + «Artista del álbum» de cada canción (`albumKeyOf`), junta «(Disc 2)» con su disco y quita duplicados. El índice de todas tus canciones se pide de a 1000 (tres páginas a la vez), se guarda cifrado y se renueva cada 10 minutos. Los ids de estos álbumes empiezan con `m` (`albumGid`).

Búsqueda en las letras (LYRX)

Jellyfin no busca dentro de las letras: Nanuq indexa las canciones con `HasLyrics` (de a 3, hasta 8000) y las letras locales, normalizadas sin acentos ni signos (`lyrNorm`). Se busca con 4 caracteres o más.

07

DJ Nanuq

Rondas con reglas, sin IA.



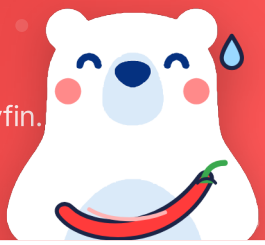
`DJ.next()` ordena los tipos de ronda al azar según su peso, evita repetir los recientes y prueba hasta encontrar una que **explore** (canciones que no sonaron en las últimas semanas). Cada canción lleva su ronda en el id de la cola: `dj—<sesión>-<ronda>-<azar>`.

Ronda	Peso	Qué pone
genre	6	Un género de tu biblioteca (≥ 3 canciones en una muestra de 400), sin repetir los recientes.
decade	3.5	Una década real de tu biblioteca; las pasadas pesan ×1.5.
discover	3	Nunca escuchadas, primero en tus géneros favoritos.
forgotten	3	Más de 45 días sin sonar.
mood	1	Según la hora (mañana tranquila, tarde con energía…).
fresh · fav · recent · top	.8 · .6 · .15 · .15	Recién llegadas, favoritas, lo de estos días y lo más escuchado (con enfriamiento).

`DJ.rank()` decide qué tan «gastada» está una canción: lo de los últimos 3 días o de sesiones recientes del DJ casi no entra; lo olvidado va primero. Al pasar la música a otro dispositivo, `DJSHARE` deja el estado en tu Jellyfin y el otro lo adopta (`DJ.adopt`); lo que se lee se limpia con `djClean`. Mientras se arma la primera ronda, `DJL` muestra a Nanuq en su tornamesa con frases (`DJ_LOAD_SAYS`) hasta que suena.

08 Mis dispositivos

Un Nanuq suena y los demás lo controlan, a través de Jellyfin.





No se abre ningún puerto en los dispositivos: todo pasa por tu Jellyfin, con tu sesión.

- **Quién aparece**: sesiones con cliente «Nanuq», de tu mismo usuario, activas hace menos de 15 minutos según el reloj del *servidor* (los relojes desfasados no estorban).
- **Misma red**: misma /24 en IPv4 o /64 en IPv6 (o ambas privadas), con la dirección que ve el servidor.
- **Órdenes aceptadas**: `Play` y `GeneralCommand` solo si las manda tu usuario (dato que pone Jellyfin); `Playstate` lo filtra Jellyfin por permisos.
- **Controlando**, tu cola local se aparta (`RC.localQ`) y vuelve al desconectarte; la posición se estima desde el último reporte (`RC.anchor`).
- En Android, con la pantalla bloqueada, la app nativa llama a `__nanuqTick` cada 20 s para mantener vivo el WebSocket.

09

La fiesta

Teléfonos que mandan canciones a una computadora por la red local.





UDP :47865 «Who is NanuqParty?» → nombre y puerto de la sala

El teléfono baja la canción de SU Jellyfin y se la pasa a la computadora en trozos de 256 KB, sin tenerla entera en memoria.

Mensaje	Sentido	Qué es
hello {code|key, name, theme}	teléfono → PC	Entrar con el código de 6 dígitos o volver con su llave.
welcome {you, key, room, chunk}	PC → teléfono	Entró: su llave para volver sin código (2 h).
add · [id + bytes] · end · remove	teléfono → PC	Mandar una canción (datos, letra, carátula) y sus bytes.
addOk · addErr · ready	PC → teléfono	Respuesta al pedido y aviso de que ya está lista.
state · tick	PC → teléfonos	Lista, invitados y lo que suena; segundo y línea del karaoke.
bye {why} · leave · ping	ambos	Salir, ser sacado o avisar que sigue ahí.

Topes y defensas

`PT_MAX` en `main.js`: `{ guests: 12, conns: 24, file: 80e6, disk: 800e6, list: 60, perGuest: 10, frame: 1 << 20, text: 450e3, lyrics: 200e3, cover: 150e3 }`. Solo direcciones privadas (`ptLocal`), ≤ 4 conexiones sin presentarse por dirección, 5 códigos malos por minuto por dirección y, con 30 fallos en total, código nuevo y un minuto cerrado. El JSON se lee con `jparse` y los textos con `ptStr`; un mensaje que haga fallar algo solo corta esa conexión.

Canciones en disco, cifradas

Las canciones van a una carpeta en disco (`ptBase()`: `~/.cache/nanuq/nanuq-fiesta-*` en Linux; en el Flatpak, dentro de `~/.var/app/com.nanuq.app/cache`), no a la temporal, que allí es memoria RAM. Se

rechaza una canción si quedan menos de 1 GB libres. Cada canción se guarda **cifrada y autenticada** con AES-256-GCM en bloques de 64 KB (`PT_CH`): la clave es al azar y solo vive en la memoria de la sala (`PT.fkey`), y el nonce es de 8 bytes al azar por canción más el número de bloque (`ptNonce`). `ptSeal` cifra al llegar; `ptServe` comprueba el tamaño exacto (`ptEncSize`) y `ptOpenRange` descifra solo los bloques del rango pedido. Si otra app cambia un byte, corta el archivo o cambia bloques de lugar, el bloque no se entrega y la canción sale de la lista.

10 La app de Android

WebView + Java nativo.

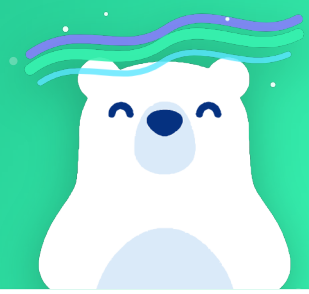


Clase	Responsabilidad
MainActivity · Page	`MainActivity` es solo la pantalla (Atrás, color de las barras). La página vive en `Page` : WebView (assets en `http://localhost`), el puente `NanuqAndroid` , latido de 20 s, recrear el WebView si muere, enlaces externos solo http(s). Puede arrancar sin pantalla (`resumeHeadless` , con `?resume=1`) y la pantalla la adopta después (`MutableContextWrapper`).
NativeAudio	La música: un `MediaPlayer` por reproductor, foco de audio propio (pérdida pasajera: pausa y vuelve; otra app: pausa y vuelve cuando ya no suena nada, con `isMusicActive` , hasta 5 min), rampas sin chasquidos al empezar, pausar y soltar. Avisos a la página con un número de canción (`gen`) para ignorar los viejos.
MediaCenter · PlaybackService	MediaSession y notificación; servicio en primer plano con bloqueos de CPU y Wi-Fi mientras suena; en pausa sigue (sin bloqueos) para que no desaparezcan los controles, hasta que deslizas la notificación o cierras la app. ▶ sin página arranca la página sin pantalla.
NanuqBrowserService · Resume · MediaButtonReceiver	«Reanudar» con la app cerrada: el servicio (solo responde a apps del sistema) da la última canción a la interfaz de Android; el receptor (permiso `MEDIA_CONTENT_CONTROL` , solo el sistema) recibe el ▶ de los audífonos.
Downloads	Descargas de una en una, solo Wi-Fi por omisión, límite con reemplazo de automáticas; índice cifrado; Opus solo si sus páginas Ogg están completas (`OggCheck`).
Prefetch	Lo que sigue en la cola, en la caché (≤ 400 MB por archivo, 1 GB total, 300 MB libres).
LocalServer	HTTP en 127.0.0.1 con token: `a/` audio, `c/` carátula, `p/` por adelantado; Range; 10 s por petición, ≤ 100 encabezados, 32 hilos.
SecureStore	Almacén AES-GCM con clave en Android Keystore; estados locked / lost; los índices grandes en su propio archivo.
NativeHttp · Discovery	HTTP sin CORS solo para las rutas de SomeDL; búsqueda UDP de Jellyfin y de fiestas.

El puente `android-bridge.js` se inyecta en `index.html` al compilar: reemplaza `navigator.mediaSession` por uno que habla con `MediaCenter` y define `NanuqNativeAudio` (se usa como un `<audio>` ; con `blob:` o `data:` , el modo demo, usa un `<audio>` por dentro). Solo se descargan direcciones con forma de Jellyfin (`Downloads.okUrl`) y las redirecciones solo dentro del mismo servidor (`openSameHost`).

11 La app de escritorio

Electron para Linux y Windows.



Canal IPC	Qué hace
nanuq:discover	Busca servidores Jellyfin por UDP.
nanuq:http	HTTP sin CORS, solo rutas de SomeDL, sin redirecciones, ≤ 4 MB.
nanuq:sec (sync) · nanuq:secreset	Almacén cifrado: clave AES-256 aleatoria guardada con `safeStorage` (llavero / DPAPI); los índices grandes en su propio archivo (`SEC_BIG`).
nanuq:discord	Rich Presence por el socket `discord-ipc-N` (solo sockets tuyos), con reintento mínimo y estado para Ajustes.
nanuq:party · nanuq:party-ev	Abrir/cerrar la sala y operarla (jump, order, played, now, tick, lyrics).

- Todo canal IPC comprueba `fromApp(e)` : solo la página de nuestro origen puede pedirlo.
- Ventana con `contextIsolation` , `sandbox` y sin Node; permisos solo notificaciones, pantalla completa y portapapeles.
- Fusibles de Electron: sin `runAsNode` ni `NODE_OPTIONS` , integridad del asar y solo cargar desde el asar.
- **Integridad del código:** sin caché HTTP en disco (`disable-http-cache` ; la carpeta `Cache` vieja se borra al arrancar) y, antes de abrir la página, se borran el service worker, Cache Storage y la caché de código de V8 (también al recargar con F5). `window.json` solo acepta números razonables.
- Servidor local: solo GET/HEAD, `Host` 127.0.0.1/localhost (contra DNS rebinding), rutas ≤ 512 y sin bytes nulos.

12 Seguridad

Lo que llega de afuera nunca se cree.



- **CSP** sin scripts en línea, sin `eval`, sin iframes ni `<base>`; solo atributos `style`.
- **JSON de afuera** con `jparse` (sin `toString` / `valueOf` / `__proto__`) y textos con `str()`: nunca `String(x)` sobre datos ajenos.
- **HTML**: todo dato pasa por `esc()`; los ids por `sid()` (solo letras, números y guiones) antes de ir a una ruta.
- **Letras** limpias de control y etiquetas (`cleanLyricsText`), ≤ 200 000 caracteres.
- **Puentes nativos** con lista blanca de rutas y topes de tamaño; nada de seguir redirecciones a otro servidor.
- **Datos personales** cifrados en disco; respaldo de Android desactivado (`allowBackup=false` y reglas de extracción).
- **Android, lo que otras apps pueden tocar**: solo el servicio de «reanudar» (responde únicamente a apps del sistema) y el receptor de botones (exige `MEDIA_CONTENT_CONTROL`, que solo tiene el sistema). El reproductor nativo solo abre direcciones http(s).
- **Integridad en disco**: nada de lo guardado se usa sin comprobar. Lo cifrado (datos personales, índice de descargas, canciones de la fiesta) va con AES-GCM, que detecta cualquier cambio; las preferencias sin cifrar se validan al leerlas; en Android todo vive en el almacenamiento privado de la app y una descarga Opus solo se guarda si todas sus páginas Ogg llegan completas con su CRC (`OggCheck.valid`; las conversiones no traen tamaño); el tipo de audio del servidor solo se acepta como `audio/…` (`OggCheck.mime`).



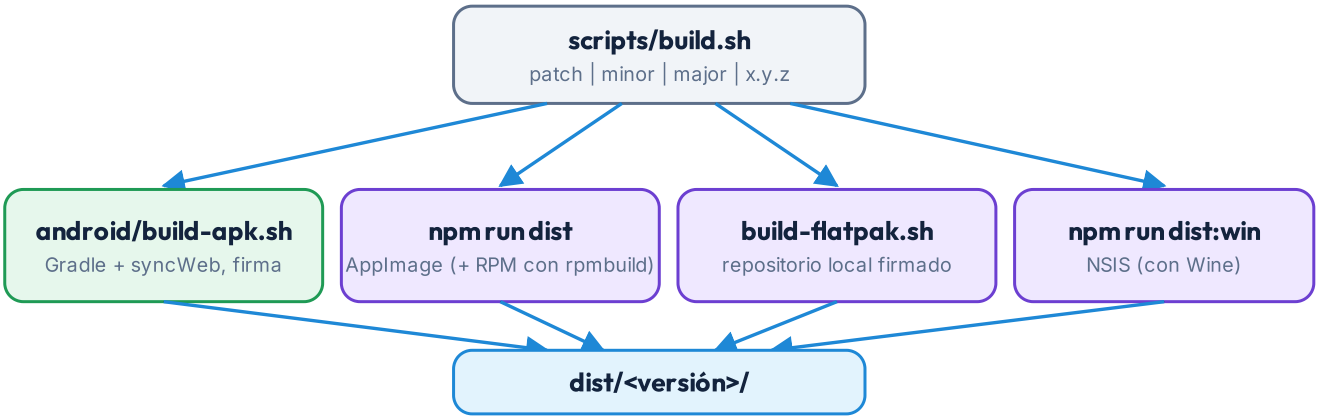
Consejo de Nanuq

El detalle completo (qué se protege y los límites conocidos) está en `SECURITY.md`.

13 Compilar y publicar

Una orden para todas las plataformas.





La versión vive en `web/app.js` (VERSION) y en `web/sw.js` (CACHE); el APK, el package.json y el Flatpak la toman de ahí.

- **Versionado**: funciones nuevas suben el número menor; arreglos, el último. El `versionCode` del APK es M·10000 + m·100 + p.
- **Android** necesita JDK 21, el SDK, Node.js y `android/keystore.properties`. Firmado con la misma llave siempre.
- **Linux**: el RPM usa el `fpm` del sistema (`USE_SYSTEM_FPM=true`) y solo se arma si hay `rpmbuild`. El Flatpak se arma con `linux-unpacked`.
- **Pruebas automáticas** (sin dependencias; si fallan, no hay paquetes): `android/test` (OggCheck y el reproductor del puente, en `build-apk.sh`) y `linux/test` (cifrado de la fiesta, quién entra a la sala y almacén cifrado, en `build.sh`).
- **Comprobar**: `node --check web/app.js` y, para interfaz, Chromium en modo demo.

14 Convenciones y trampas

Lo que conviene saber antes de cambiar algo.



Regla	Por qué
Cambios de interfaz solo en `web/`	Android y Linux la copian al compilar.
Todo texto visible en `tx('…')` + su traducción en `EN`	Si no, sale en español con el idioma en inglés. Una frase completa por `tx`, sin pegar trozos.
Nada de `onclick=`, `<script>` ni `<style>` desde JS	La CSP los bloquea sin avisar.
Ids únicos en toda la app	Un id repetido escondió el cuadro de Buscar (1.30.2).
`view-transition-name` únicos	Si se repiten, la transición se cancela sin error visible.
Animaciones sin fin solo en ráfagas (`nqBurst`)	A 120 Hz cada una cuesta ≈ 1 núcleo del teléfono.
Respetar `prefers-reduced-motion`	Accesibilidad.
Tras escribir en Jellyfin, mostrar lo local y confirmar después	La lectura inmediata puede devolver lo viejo (letras, playlists, favoritas).
`.ig-crowd` sin `z-index`	Crearía su propia capa y todos quedarían detrás del DJ.
La regla de ProGuard apunta a `Page$Bridge`	Si el puente cambia de clase y la regla no, el APK minimizado borra los métodos y la página queda sin puente.
Probar en el teléfono: `am stop-app`, no `force-stop`	`force-stop` deja la app «detenida» y Android no le entrega los botones multimedia; `am kill` no puede con el servicio en primer plano.

15 Licencia

GPLv3 o posterior, archivo por archivo.



Nanuq es software libre bajo la **GNU General Public License, versión 3 o (a elección de quien lo reciba) cualquier versión posterior**: `GPL-3.0-or-later`. El texto completo está en `LICENSE`, en la raíz del repositorio.

Qué significa para quien lo modifica

- Puedes usar, estudiar, copiar y modificar el código para lo que quieras.
- Si **distribuyes** Nanuq o una versión modificada (un APK, un paquete, la PWA en un servidor que otros descargan), debe ir bajo la misma licencia y con su código fuente (o una oferta para obtenerlo), con los avisos de copyright y de licencia intactos.
- Una versión modificada debe decir que lo es. Para Android, usa otro `applicationId` y tu propia llave de firma: el APK oficial solo lo firma su autor (ver «Compilar y publicar»).
- Se ofrece **sin ninguna garantía**.

Cómo está marcado el código

Cada archivo fuente empieza con dos líneas en el formato de **SPDX** (el que usan las herramientas de licencias y la especificación REUSE):

```
SPDX-FileCopyrightText: 2026 Fleyrer y colaboradores de Nanuq
SPDX-License-Identifier: GPL-3.0-or-later
```

Van como comentario del lenguaje de cada archivo (`//` , `/* */` , `<!-- -->` o `#`), después del `#!` en los scripts y del `<!DOCTYPE>` en el HTML. Al crear un archivo nuevo, agrégale las mismas dos líneas.

Dónde más aparece

Lugar	Cómo
`LICENSE`	Texto oficial de la GPLv3 (GitHub lo detecta y lo muestra en el repositorio).
README.md · README.en.md	Línea arriba, sección «Licencia» y enlace en el menú.
Ajustes de la app	Fila «Nanuq x.y.z»: la licencia y un enlace a `LICENSE` (se abre en el navegador).
linux/package.json	`"license": "GPL-3.0-or-later"`
Flatpak (metainfo)	`<project_license>GPL-3.0-or-later</project_license>`
Estos manuales	Portada y este capítulo.

Lo que no es de Nanuq

Componente	Licencia	Dónde
Electron	MIT	Dentro de las apps de escritorio (Linux y Windows).
Inter · Outfit	SIL Open Font License	Fuentes, cargadas de Google Fonts (no van en el repositorio).
Gradle wrapper	Apache 2.0	`android/gradlew`
Jellyfin	GPL-2.0	El servidor: un proyecto aparte con el que Nanuq habla por su API. Nanuq no está afiliado a él.

PARTE 2

Nivel profundo: cada función

Las 895 funciones y métodos del código, archivo por archivo, con la línea donde empiezan y qué hacen.



16 web/app.js

604 funciones, en el orden del archivo.



web/app.js 6128 líneas

Base: ayudantes y almacén desde la línea 1 · 25

Función	Línea	Qué hace
$	5	Atajo de document.querySelector (opcionalmente dentro de otro elemento).
$$	6	Atajo de querySelectorAll que devuelve un arreglo.
jparse	10	Lee JSON de afuera sin toString/valueOf/__proto__.
str	11	Convierte cualquier valor a texto sin lanzar nunca un error.
esc	12	Escapa un texto para meterlo en HTML (& < > " ').
sid	16	Limpia un id del servidor: solo letras, números y guiones.
num	17	Convierte a número; si no es finito, 0.
numOrNull	18	Convierte a número o devuelve null si está vacío o no es válido.
ic	30	HTML de un ícono SVG de las <defs> (#i-nombre).
fmt	31	Formatea segundos como m:ss o h:mm:ss.
isSensitive	38	¿Esta clave guardada contiene datos personales (va al almacén cifrado)?
SEC.ok	46	¿El almacén cifrado nativo está abierto y usable?
SEC.shut	47	¿El almacén está bloqueado o perdido (no se escribe nada en claro)?
store.get	55	Lee un valor guardado (cifrado si es sensible, si no localStorage) y migra copias viejas en claro.
store.set	67	Guarda un valor; lo sensible va al almacén cifrado o, si está cerrado, solo a memoria.
store.del	78	Borra un valor guardado en el almacén y en localStorage.
secMigrate	81	Pasa al almacén cifrado lo que versiones anteriores dejaron en claro.
Vault.kind	95	Nombre del almacén cifrado en uso (o null).
Vault.init	98	Arranque en frío: reintenta abrir el almacén unos segundos antes de rendirse.
Vault.save	101	Guarda la sesión de Jellyfin.

Función	Línea	Qué hace
`Vault.load`	102	Lee la sesión guardada (o marca que el almacén está bloqueado).
`Vault.reset`	109	«Empezar de cero»: aparta los datos ilegibles y crea un almacén nuevo.
`uid`	117	Id aleatorio (UUID sin guiones).
`deviceId`	118	Id persistente de este dispositivo para Jellyfin.
`deviceName`	119	Nombre del sistema (Android, Linux, Windows…) que se anuncia a Jellyfin.

Direcciones del servidor desde la línea 122 · 1

Función	Línea	Qué hace
`parseServer`	124	Interpreta lo que escribiste como servidor (IPv4, IPv6, nombre, puerto y ruta) y arma la URL base.

API de Jellyfin desde la línea 177 · 58

Función	Línea	Qué hace
`Jellyfin.constructor`	180	Crea el cliente de Jellyfin a partir de la sesión guardada.
`Jellyfin.authHeader`	181	Arma la cabecera de autorización MediaBrowser con dispositivo, versión y token.
`Jellyfin.req`	185	Petición a la API con tiempo límite, sin caché; detecta 401 y lee el JSON con jparse.
`Jellyfin.alt`	203	Prueba varias rutas de la API hasta que una no dé 404.
`Jellyfin.probe`	204	Comprueba que una dirección sea un Jellyfin (/System/Info/Public).
`Jellyfin.login`	212	Inicia sesión con usuario y contraseña y guarda el token.
`Jellyfin.cfg`	216	Datos de la sesión para guardar.
`Jellyfin.map`	217	Convierte un elemento de Jellyfin al formato de Nanuq, limpiando ids, números y textos.
`Jellyfin.items`	236	Consulta genérica de /Items con los campos que usa la app.
`Jellyfin.albums`	240	Álbumes de Jellyfin paginados y ordenados.
`Jellyfin.item`	244	Un elemento por id (con ruta de respaldo para servidores viejos).
`Jellyfin.album`	245	Un álbum de Jellyfin y sus pistas.
`Jellyfin.artists`	249	Artistas del álbum, paginados.
`Jellyfin.artistFilters`	254	Filtros posibles para encontrar las cosas de un artista (por varios ids o por nombre).
`Jellyfin.firstNonEmpty`	260	Prueba filtros en orden y devuelve el primer resultado que no esté vacío.

Función	Línea	Qué hace
`Jellyfin.artist`	264	Un artista y sus álbumes (armados con sus canciones si hace falta).
`Jellyfin.artistTracks`	279	Todas las canciones de un artista.
`Jellyfin.genres`	287	Géneros de música probando varias rutas; si no, los saca de los álbumes.
`Jellyfin.genreParam`	306	Parámetros de consulta para uno o varios géneros (por id o por nombre).
`Jellyfin.genre`	311	Álbumes de un género.
`Jellyfin.genreTracks`	318	Canciones de un género (de las etiquetas de las canciones o de sus álbumes).
`Jellyfin.playlists`	327	Tus playlists de audio.
`Jellyfin.playlist`	328	Una playlist y sus canciones.
`Jellyfin.views`	332	Bibliotecas de música del usuario (raíz de Carpetas).
`Jellyfin.folder`	336	Contenido de una carpeta.
`Jellyfin.search`	337	Búsqueda de artistas, álbumes y canciones.
`Jellyfin.img`	344	URL de la carátula (local si está descargada en el teléfono).
`Jellyfin.streamUrl`	351	URL de streaming universal: formatos que suenan tal cual y conversión a Opus o MP3.
`Jellyfin.stream`	357	URL para reproducir: descarga, caché por adelantado o streaming.
`Jellyfin.dlUrl`	361	URL para descargar una canción al teléfono (la de streaming con la calidad de las descargas: Opus o MP3, o el archivo tal cual).
`Jellyfin.peaks`	362	Baja una copia ligera y calcula la forma de onda real.
`Jellyfin.tracks`	367	Canciones de la biblioteca paginadas y ordenadas.
`Jellyfin.lyrics`	371	Letra de una canción (Jellyfin 10.9+), normalizada.
`Jellyfin.mix`	376	Canciones para una mezcla (orden, filtros, géneros, años).
`Jellyfin.uploadLyrics`	383	Sube una letra (.lrc o .txt) a Jellyfin.
`Jellyfin.searchLyrics`	390	Busca letras en los proveedores de Jellyfin (LrcLib).
`Jellyfin.pickLyrics`	394	Elige una de las letras encontradas para guardarla.
`Jellyfin.setFavorite`	396	Marca o quita una favorita.
`Jellyfin.inFavorites`	402	¿Está en la sección de favoritas de Jellyfin?
`Jellyfin.isFavorite`	403	¿Esta canción es favorita según sus datos de usuario?
`Jellyfin.favPlaylist`	410	Encuentra la playlist del plugin «Favorited Songs Playlist».
`Jellyfin.favorites`	417	Tus favoritas (filtro IsFavorite), paginadas y ordenadas.
`Jellyfin.setFavoriteLegacy`	435	Ruta antigua para guardar favoritas (segundo intento).

Función	Línea	Qué hace
`Jellyfin.instantMix`	437	Mezcla instantánea de Jellyfin a partir de una canción (radio).
`Jellyfin.addToPlaylist`	442	Agrega canciones a una playlist.
`Jellyfin.refreshLibrary`	444	Pide a Jellyfin que vuelva a escanear las bibliotecas (administradores).
`Jellyfin.savePlaylist`	445	Crea una playlist nueva con canciones.
`Jellyfin.report`	446	Avisa a Jellyfin que empezó, avanza o terminó una canción (y reafirma favoritas).
`Jellyfin.logout`	452	Cierra la sesión en el servidor (revoca el token).
`Jellyfin.wsUrl`	455	URL del WebSocket de Jellyfin.
`Jellyfin.postCapabilities`	456	Anuncia que este Nanuq acepta órdenes de control remoto.
`Jellyfin.sessions`	458	Sesiones activas (para Mis dispositivos).
`Jellyfin.sessionCmd`	459	Manda una orden de reproducción a otra sesión.
`Jellyfin.sessionPlay`	460	Manda una cola para reproducir en otra sesión.
`Jellyfin.sessionGeneral`	461	Manda una orden general (repetir, mezclar) a otra sesión.
`Jellyfin.itemsByIds`	462	Pide varios elementos por id, de 100 en 100.
`netError`	468	Mensaje claro cuando no hay conexión con el servidor.
`normLyrics`	477	Normaliza una letra de Jellyfin (sincronizada o en texto).

Audio: formas de onda y sintetizador

desde la línea 489 · 6

Función	Línea	Qué hace
`actx`	492	AudioContext compartido (creado al primer uso).
`decode`	493	Decodifica audio a un AudioBuffer.
`peaksFromBuffer`	494	Calcula los picos de la forma de onda de un AudioBuffer.
`rng`	501	Generador pseudoaleatorio con semilla.
`wav`	502	Convierte un AudioBuffer en un archivo WAV.
`synth`	509	Sintetiza una canción de prueba para el modo demo.

Biblioteca demo

desde la línea 550 · 43

Función	Línea	Qué hace
`Demo.constructor`	553	Crea la biblioteca de demostración (artistas, álbumes, canciones y playlists).
`Demo.cfg`	588	Sesión del modo demo.
`Demo.wait`	589	Simula la espera de la red.
`Demo.albums`	590	Versión demo de Jellyfin.albums.
`Demo.album`	595	Versión demo de Jellyfin.album.

Función	Línea	Qué hace
`Demo.artists`	596	Versión demo de Jellyfin.artists.
`Demo.artistBy`	598	Busca un artista demo por id o por nombre.
`Demo.artist`	599	Versión demo de Jellyfin.artist.
`Demo.artistTracks`	600	Versión demo de Jellyfin.artistTracks.
`Demo.genres`	601	Versión demo de Jellyfin.genres.
`Demo.genreTracks`	602	Versión demo de Jellyfin.genreTracks.
`Demo.genre`	603	Versión demo de Jellyfin.genre.
`Demo.playlists`	604	Versión demo de Jellyfin.playlists.
`Demo.playlist`	605	Versión demo de Jellyfin.playlist.
`Demo.views`	606	Versión demo de Jellyfin.views.
`Demo.folder`	607	Versión demo de Jellyfin.folder.
`Demo.search`	612	Versión demo de Jellyfin.search.
`Demo.img`	616	Dibuja una carátula SVG generada para el modo demo.
`Demo.buf`	630	Audio sintetizado de una canción demo (en caché).
`Demo.stream`	631	URL blob del WAV de una canción demo.
`Demo.peaks`	632	Forma de onda de una canción demo.
`Demo.tracks`	633	Versión demo de Jellyfin.tracks.
`Demo.mix`	639	Versión demo de Jellyfin.mix.
`Demo.lyrics`	649	Letra sincronizada inventada para una canción demo.
`Demo.setFavorite`	661	Versión demo de Jellyfin.setFavorite.
`Demo.isFavorite`	662	Versión demo de Jellyfin.isFavorite.
`Demo.inFavorites`	663	Versión demo de Jellyfin.inFavorites.
`Demo.favorites`	664	Versión demo de Jellyfin.favorites.
`Demo.instantMix`	665	Versión demo de Jellyfin.instantMix.
`Demo.addToPlaylist`	666	Versión demo de Jellyfin.addToPlaylist.
`Demo.savePlaylist`	667	Versión demo de Jellyfin.savePlaylist.
`Demo.report`	668	No hace nada (el demo no reporta reproducciones).
`Demo.logout`	669	No hace nada (sin servidor).
`logoInner`	674	HTML del oso del logo con el accesorio del tema.
`bearHTML`	690	HTML del Nanuq animable (logo + cara expresiva).
`bearState`	692	Estado actual del oso (bailando, contento, triste) para los osos nuevos.
`nqBurst`	698	Activa una ráfaga de animaciones por unos segundos (ahorro de batería).

Función	Línea	Qué hace
nqBurstEnd	706	Termina la ráfaga: las animaciones se pausan y los osos quedan contentos.
bearDance	711	Hace bailar a todos los Nanuqs un rato.
bearInit	716	Pone el Nanuq en los logos de la barra lateral.
bearMood	721	Cambia el ánimo de los osos al sonar o pausar la música.
showSplash	732	Pantalla de bienvenida con el Nanuq del tema.
refreshBear	743	Cambia el accesorio de los osos al cambiar de tema.

Ajustes y temas desde la línea 748 · 3

Función	Línea	Qué hace
OPUS_OK	752	¿El reproductor abre Opus? Decide si se ofrece y si va por omisión.
newAudio	755	Crea el reproductor: en Android, el del sistema (NanuqNativeAudio); si no, un <audio>.
Settings.set	785	Cambia un ajuste y lo guarda.

Descargas (Android) desde la línea 808 · 30

Función	Línea	Qué hace
DL.init	814	Carga las descargas del teléfono desde la app nativa.
DL.indexCovers	818	Índice de carátulas descargadas por id de imagen.
DL.coverUrl	826	URL local de una carátula descargada.
DL.backfillCovers	828	Completa carátulas y datos de descargas viejas o recuperadas.
DL.config	819	Manda a Android el máximo de canciones y si solo con Wi-Fi.
DL.usable	829	¿Hay descargas en este dispositivo (Android con Jellyfin)?
DL.get	826	La descarga de una canción, si existe.
DL.localUrl	838	URL local de una canción descargada y lista.
DL.isLocal	839	¿Esta URL es del servidor local del teléfono?
DL.add	833	Pide descargar una canción (manual o automática).
DL.remove	846	Quita una canción de las descargas.
DL.removeAll	847	Borra todas las descargas (o solo las automáticas).
DL.list	848	Lista de descargas.
DL.bytes	849	Espacio total que ocupan las descargas.
DL.net	850	Tipo de red actual según Android (wifi, datos, ninguna).
__nanuqDl	852	Recibe de Android los cambios de una descarga y actualiza la vista.

Función	Línea	Qué hace
`Plays.count`	867	Veces que sonó una canción en este teléfono.
`Plays.add`	868	Cuenta una reproducción y guarda sola la canción a las 3.
`PF.usable`	874	¿Se puede guardar por adelantado (Android con Jellyfin)?
`PF.isUrl`	886	¿Esta URL es de lo guardado por adelantado?
`PF.url`	887	URL local de una canción ya guardada por adelantado (o null).
`PF.soon`	888	Programa una actualización de la ventana por adelantado.
`PF.sync`	888	Calcula qué canciones de la cola guardar (N minutos) y se lo pide a Android.
`fmtMB`	904	Formatea bytes como MB o GB según el idioma.
`dlErr`	907	Traduce un error de descarga de Android (el primer número va como {0}).
`dlStatus`	911	Texto del estado de una descarga.
`dlStatusShort`	922	Estado corto de una descarga para el botón del reproductor.
`dlSummary`	928	Resumen de las descargas (cantidad, espacio, pendientes).
`themeSummary`	952	HTML del tema elegido en Ajustes.
`applyTheme`	956	Aplica un tema (colores, barra del sistema, osos).

Estado y navegación desde la línea 967 · 8

Función	Línea	Qué hace
`regList`	973	Registra una lista de canciones para que los botones la encuentren.
`cur`	982	Pantalla actual de la pila de navegación.
`go`	983	Navega a una pantalla (raíz o encima de la actual).
`back`	992	Vuelve atrás: cierra diálogos, el reproductor o la pantalla actual.
`cacheDrop`	1002	Olvida las vistas en caché que empiezan con un prefijo.
`refreshIf`	1004	Redibuja la pantalla actual si cumple una condición.
`playlistsChanged`	1007	Olvida y vuelve a pedir una playlist que cambió.
`cached`	1014	Caché de vistas con tope (las más viejas salen primero).

Álbumes según las etiquetas desde la línea 1017 · 15

Función	Línea	Qué hace
`albumNorm`	1023	Normaliza un nombre para comparar álbumes.
`albumKeyOf`	1025	Clave del álbum de una canción según sus etiquetas (artista + álbum + disco).
`albumGid`	1032	Id estable de un álbum armado por etiquetas.

Función	Línea	Qué hace
`LIB.who`	1035	Identifica servidor y usuario del índice.
`LIB.index`	1037	Índice de álbumes por etiquetas (desde caché cifrada o del servidor).
`LIB.peek`	1050	El índice si ya está listo, sin esperar.
`LIB.reset`	1051	Marca el índice como viejo para renovarlo.
`LIB.refresh`	1045	Vuelve a armar el índice con todas las canciones.
`LIB.fetchAll`	1053	Baja todas las canciones de la biblioteca (3 páginas a la vez).
`LIB.pack`	1068	Versión compacta de una canción para guardar el índice.
`LIB.unpack`	1069	Lee una canción compacta del índice guardado, limpiándola.
`LIB.save`	1074	Guarda el índice cifrado (si cabe).
`LIB.build`	1079	Agrupa las canciones en álbumes, sin repetidas.
`LIB.albumsOf`	1104	Álbumes (por etiquetas) de un grupo de canciones.
`LIB.sorted`	1109	Lista de álbumes ordenada.

Búsqueda en las letras desde la línea 1122 · 10

Función	Línea	Qué hace
`lyrNorm`	1128	Normaliza texto de letras para buscar (sin acentos ni signos).
`LYRX.load`	1131	Carga el índice de letras guardado.
`LYRX.add`	1136	Agrega la letra de una canción al índice.
`LYRX.save`	1141	Guarda el índice de letras cifrado (con espera).
`LYRX.reset`	1149	Borra el índice de letras.
`LYRX.build`	1151	Arma o completa el índice leyendo letras de a 3.
`LYRX.search`	1177	Busca canciones cuya letra contiene un texto.
`lyrSnippetHTML`	1194	Línea de letra con lo buscado resaltado (escapada).
`lyricsHitsHTML`	1199	Lista HTML de canciones encontradas por la letra.
`lyricsSection`	1204	Sección «En las letras» que se completa mientras se indexa.

Ayudantes de dibujo desde la línea 1220 · 8

Función	Línea	Qué hace
`coverHTML`	1222	HTML de una carátula con su ícono de respaldo.
`head`	1227	Encabezado de una pantalla (título, botones, oso, atrás).
`albumCard`	1231	Tarjeta de un álbum.
`artistCard`	1232	Tarjeta de un artista.
`trackRows`	1233	Contenedor de filas de canciones.
`rowsHTML`	1233	Filas de canciones (número, título, duración, menú).

Función	Línea	Qué hace
loading	1246	Indicador de carga.
errBox	1247	Caja de error con «Reintentar».

Vistas desde la línea 1250 · 6

Función	Línea	Qué hace
V.albums	1253	Pantalla Álbumes (por etiquetas, de 120 en 120).
V.album	1259	Pantalla de un álbum.
normTxt	1299	Normaliza texto (sin acentos, minúsculas).
mixDef	1300	Definición de una mezcla (consulta según géneros o años).
mixTracks	1314	Canciones de una mezcla (en caché).
greeting	1315	Saludo según la hora.

DJ Nanuq desde la línea 1317 · 85

Función	Línea	Qué hace
djBearSVG	1323	SVG de DJ Nanuq con audífonos, bufanda y tornamesa.
djMoment	1375	Momento del día y ánimo para la ronda «Para este momento».
pick	1376	Elige uno al azar.
wpick	1378	Elige al azar con pesos.
wshuffle	1380	Ordena al azar dando más chance a lo que pesa más.
djQid	1383	Id de cola que lleva la sesión y la ronda del DJ.
djTag	1384	Lee sesión y ronda de un id de cola del DJ.
djSame	1385	¿Dos ids son de la misma sesión del DJ?
DJ.voiceOk	1388	¿Hay una voz del sistema en el idioma de la app?
DJ.save	1389	Guarda la sesión del DJ y la comparte con tus dispositivos.
DJ.restore	1394	Recupera la sesión del DJ al abrir la app.
DJ.sample	1401	Muestra al azar de la biblioteca para elegir géneros y décadas.
DJ.heavy	1407	Tus 60 canciones más escuchadas.
DJ.pickGenre	1413	Elige el género de una ronda (sin repetir los recientes).
DJ.pickDecade	1423	Elige la década de una ronda.
DJ.rank	1435	Qué tan gastada está una canción (más bajo = va antes).
DJ.pool	1444	Canciones candidatas para un tipo de ronda.
DJ.build	1477	Arma una ronda de ~5 canciones con su presentación.
DJ.next	1504	Elige y arma la siguiente ronda (pesos y enfriamiento).
DJ.items	1518	Convierte una ronda en canciones de la cola y la anota.

Función	Línea	Qué hace
`DJ.lastKey`	1520	Número de la última ronda.
`DJ.start`	1527	Empieza una sesión del DJ con su pantalla de carga (aquí o en el dispositivo que controlas).
`DJ.startNow`	1530	Arma la primera ronda y espera a que suene, como mucho 12 s (detrás de la pantalla de carga).
`DJ.share`	1545	Deja en Jellyfin el estado del DJ para otro dispositivo.
`DJ.handedOff`	1546	La sesión pasó a otro dispositivo: aquí se apaga.
`DJ.adopt`	1565	Sigue aquí una sesión que viene de otro dispositivo.
`DJ.queueFromShare`	1570	Reconstruye la cola del DJ desde el estado compartido.
`DJ.queueFromTags`	1572	Reconstruye la cola del DJ desde los ids de la cola.
`DJ.remoteTag`	1574	¿Lo que suena en el dispositivo que controlas es el DJ?
`DJ.shown`	1575	¿Hay que mostrar el DJ?
`DJ.remoteTrack`	1577	Trae las rondas del DJ que suena en otro dispositivo.
`DJ.stop`	1587	Apaga el DJ (la cola queda como está).
`DJ.onTrack`	1589	Al cambiar de canción: presenta la ronda nueva y agrega la siguiente.
`DJ.extend`	1594	Agrega la siguiente ronda al final de la cola.
`DJ.switchUp`	1607	«Cambiar de onda»: otra ronda en lugar de lo que quedaba.
`DJ.announce`	1615	Presenta una ronda (globo, aviso y voz).
`DJ.show`	1615	Muestra la presentación donde corresponda.
`DJ.speak`	1615	Lee la presentación en voz alta y baja la música.
`DJ.visible`	1640	¿La página está a la vista?
`DJ.bears`	1641	Enciende o apaga el baile de los DJ.
`DJ.say`	1645	Globo de diálogo del DJ en el reproductor.
`DJ.scene`	1650	Escena del disco en el reproductor (funda, vinilo, entradas y salidas).
`DJ.paint`	1669	Actualiza todo lo del DJ en pantalla.
`DJSHARE.ok`	1699	¿Se puede compartir el DJ (Jellyfin y sincronización activada)?
`DJSHARE.state`	1700	Estado del DJ para compartir (acotado).
`DJSHARE.push`	1705	Guarda el estado del DJ en /DisplayPreferences.
`DJSHARE.pushRaw`	1713	Guarda un estado ya armado.
`DJSHARE.pushSoon`	1717	Guarda el estado en 1.5 s.
`DJSHARE.offSoon`	1719	Avisa que el DJ se apagó, si la sesión sigue siendo de aquí.
`DJSHARE.pull`	1721	Lee el estado compartido del DJ.
`djClean`	1726	Limpia el estado del DJ leído (dato no confiable).

Función	Línea	Qué hace
djLiveSync	1744	Anima el DJ solo mientras se ve.
djTileHTML	1756	Tarjeta de DJ Nanuq en Inicio.
DJL.show	1766	Muestra la pantalla de carga del DJ con frases que rotan (un toque la quita).
DJL.hide	1770	Quita la pantalla de carga (al menos 1 s a la vista, o ya si la tocaste; una sola vez).
V.dj	1783	Pantalla de DJ Nanuq.
nanuqSceneSVG	1807	Escena de Nanuq para una celda vacía de Inicio.
nanuqBear	1823	Nanuq listo para colocar en una escena.
nanuqWideSVG	1827	Escena ancha de Nanuq (dos celdas).
nanuqWideTileHTML	1844	Celda ancha con escena.
nanuqTileHTML	1849	Celda con escena.
nanuqVisit	1856	Un Nanuq de otro tema visita una celda.
nanuqVisits	1876	Programa visitas de vez en cuando.
fillGaps	1884	Rellena los huecos de la cuadrícula de Inicio con escenas.
tileHTML	1897	Mosaico de una mezcla.
offlineBanner	1902	Aviso «Sin conexión» con acceso a Descargas.
V.home	1907	Pantalla Inicio.
V.mix	1952	Pantalla de una mezcla.
V.songs	1962	Pantalla Canciones (paginada).
V.artists	1990	Pantalla Artistas.
fmtLong	1997	Duración larga («3 h 12 min»).
V.artist	1990	Pantalla de un artista (canciones, álbumes, aparece en).
V.genres	2079	Pantalla Géneros.
V.genre	2084	Pantalla de un género.
V.playlists	2095	Pantalla Playlists.
V.playlist	2100	Pantalla de una playlist.
V.favorites	2109	Pantalla Favoritas (confirmada con Jellyfin).
V.folders	2135	Pantalla Carpetas.
V.search	2144	Pantalla Buscar (incluye la búsqueda en letras).
songsBlock	2166	Bloque de canciones en lista o cuadrícula.
V.queue	2181	Pestaña Cola (teléfono).
V.more	2182	Pantalla Más.
V.settings	2185	Pantalla Ajustes.

Función	Línea	Qué hace
`V.downloads`	2255	Pantalla Descargas.
`renderDownloadsSoon`	2271	Redibuja Descargas en un momento.

Tu año y Tu semana (video) desde la línea 2277 · 36

Función	Línea	Qué hace
`ServerStats.available`	2289	¿Está el plugin Playback Reporting y se puede leer?
`ServerStats.plays`	2297	Reproducciones entre dos fechas desde el plugin (día por día).
`Recap.total`	2359	Duración total del video.
`Recap.setMode`	2355	Cambia entre resumen del año y de la semana.
`Recap.plays`	2367	Reproducciones del periodo (plugin, datos básicos o demo).
`Recap.tally`	2377	Agrupa reproducciones: canciones, artistas, géneros, horas, días, racha.
`Recap.collect`	2401	Datos del resumen del año.
`Recap.collectWeek`	2413	Datos del resumen de la semana pasada (lunes a domingo).
`Recap.jellyfinYear`	2429	Canciones escuchadas en un periodo según los datos básicos de Jellyfin.
`Recap.assets`	2442	Prepara imágenes del video: osos, DJ, carátulas y paleta.
`Recap.loadImg`	2464	Carga una imagen con topes (8 MB, reducida) para el canvas.
`Recap.sceneAt`	2480	Qué escena toca en un segundo del video.
`Recap.frame`	2479	Dibuja un cuadro del video (con transición entre escenas).
`Recap.transition`	2488	Transición entre dos escenas (deslizar, círculo, zoom, barrido).
`Recap.bg`	2498	Fondo de una escena: degradado, brillo al ritmo y copos.
`Recap.weather`	2515	Dibuja el pronóstico musical (sol, tormenta, nieve, arcoíris…).
`Recap.bear`	2532	Dibuja un Nanuq que brinca al ritmo.
`Recap.text`	2539	Texto ajustado al ancho, con sombra.
`Recap.cover`	2549	Carátula con esquinas redondeadas.
`Recap.confetti`	2555	Lluvia de confeti.
`Recap.scene`	2563	Dibuja una escena del resumen.
`Recap.music`	2810	Música del video generada con Web Audio (con redoble).
`Recap.stop`	2841	Detiene la reproducción o grabación del video.
`Recap.play`	2840	Reproduce el video en el canvas y, si se pide, lo graba (MediaRecorder).
`Recap.save`	2868	Guarda el video (Android en trozos, escritorio como descarga).

Función	Línea	Qué hace
mp4Finalize	2894	Escribe la duración real en el MP4 fragmentado.
recapCloseAudio	2935	Cierra el audio del video una sola vez.
ease.out	2936	Curva de animación (salida).
ease.inOut	2936	Curva de animación (entrada y salida).
ease.back	2936	Curva de animación con rebote.
clamp01	2937	Limita un valor entre 0 y 1.
mixHex	2938	Mezcla dos colores hex.
hexA	2939	Color hex con transparencia.
recapYearOpen	2941	¿Es la primera semana de diciembre?
recapTileHTML	2942	Tarjeta «Tu año con Nanuq» en Inicio.
V.recap	2945	Pantalla de Tu semana / Tu año (ver y exportar).

Solicitar canción (SomeDL) desde la línea 2988 · 26

Función	Línea	Qué hace
__nanuqHttp	2994	Recibe la respuesta de una petición nativa a SomeDL (Android).
reqNative	2995	¿Hay puente nativo para hablar con SomeDL sin CORS?
reqRawHttp	2996	Petición a SomeDL por el puente nativo o fetch.
parseSomedl	3006	Interpreta la dirección de SomeDL (puerto 5000 por omisión).
reqNetError	3015	Mensaje claro cuando SomeDL no responde.
REQ.save	3030	Guarda la lista de pedidos.
REQ.setCfg	3031	Guarda o quita la dirección de SomeDL.
REQ.pending	3032	Pedidos que siguen en camino.
REQ.visible	3033	¿La app está a la vista (para revisar más seguido)?
REQ.http	3018	Petición a la API de SomeDL con manejo de errores.
REQ.probe	3042	Comprueba que una dirección sea SomeDL.
REQ.add	3048	Manda un pedido de canción a SomeDL.
REQ.schedule	3060	Programa la siguiente revisión de pedidos.
REQ.poll	3061	Revisa el estado de los pedidos en SomeDL.
REQ.announce	3090	Avisa (y notifica) qué pedidos terminaron.
REQ.retry	3105	Vuelve a pedir una canción que falló.
sysNotify	3098	Notificación del sistema (Android o escritorio).
askNotifyPermission	3118	Pide permiso de notificaciones en el navegador.
reqRowHTML	3121	Fila de un pedido de SomeDL.

Función	Línea	Qué hace
reqListHTML	3139	Lista de pedidos.
renderReqSoon	3137	Redibuja la lista de pedidos en un momento.
V.request	3141	Pantalla Solicitar canción.
refreshFromJellyfin	3201	«Actualizar desde Jellyfin»: olvida cachés, rehace índices y refresca la cola.
sessionGone	3224	Confirma si un 401 es de verdad la sesión vencida.
sessionExpired	3235	Sesión vencida: detiene todo y pide entrar.
render	3243	Dibuja la pantalla actual con transición y manejo de errores.

Reproductor desde la línea 3279 · 64

Función	Línea	Qué hace
P.pause	3289	Pausa marcando que la pidió Nanuq.
onAudio	3294	Escucha un evento en los dos <audio> pero solo atiende al de la canción actual.
saveQueue	3296	Guarda la cola y actualiza la ventana por adelantado.
QPOS.save	3301	Guarda el segundo de la canción para retomarlo.
QPOS.restore	3307	Dónde retomar la canción al abrir la app.
QPOS.clear	3311	Olvida la posición guardada.
setQueue	3315	Reemplaza la cola y empieza a reproducir.
addToQueue	3316	Agrega canciones al final o a continuación.
ptime	3326	Segundo actual de la canción (aquí o en el otro dispositivo).
pdur	3327	Duración de la canción.
setSrc	3333	Cambia la fuente del audio reiniciando su estado.
seekTo	3337	Salta a un segundo (en conversiones en vivo pide desde ahí).
playIndex	3358	Reproduce la canción N de la cola.
armStallWatch	3375	Avisa si una canción tarda demasiado en empezar.
MP3.add	3404	Recuerda una canción que debe pedirse en MP3.
localFailed	3408	La copia local falló: suena del servidor (y se vuelve a descargar).
canFallback	3420	¿Se puede pedir esta canción en MP3 (en «Original» o con Opus)?
fallbackMp3	3423	Pide la canción en MP3 si la original o la de Opus no arranca.
armFallback	3433	Programa el paso a MP3 si no arranca en 6 s.
playFailed	3439	Explica por qué no se pudo reproducir.
audioDead	3460	¿El audio quedó muerto (error de red)?

Función	Línea	Qué hace
`replayHere`	3461	Vuelve a pedir la canción desde el segundo en que iba.
`rampStop`	3474	Detiene una rampa de volumen.
`pageHidden`	3480	¿La página no está a la vista? (pantalla apagada o app en segundo plano): ahí no hay desvanecidos ni transición del DJ.
`rampsFinish`	3482	Termina ya las rampas de volumen en curso (al ocultarse la página).
`ramp`	3486	Rampa de volumen (en decibelios, o de potencia constante para el crossfade) con temporizador.
`fadeStop`	3495	Detiene un desvanecido en curso.
`fadeTo`	3496	Lleva el volumen a un valor con desvanecido.
`softMs`	3505	Duración del desvanecido de pausa/play (0 si está apagado).
`userVol`	3504	Volumen elegido por ti (bajado si habla el DJ).
`pauseSoft`	3502	Pausa con desvanecido.
`playSoft`	3510	Reanuda con desvanecido, o vuelve a subir si estaba bajando (recarga si otra app pausó).
`partyNow`	3526	¿Los controles manejan la fiesta ahora?
`partyHostOpen`	3527	¿Hay una sala de fiesta abierta en esta computadora?
`toggle`	3527	Reproducir/pausar (fiesta, control remoto o aquí).
`next`	3534	Siguiente canción (respeta repetir y el fin de la cola).
`prev`	3546	Canción anterior (o al inicio si van más de 3 s).
`seekFrac`	3547	Salta a una fracción de la canción.
`removeAt`	3553	Quita una canción de la cola.
`clearQueue`	3553	Vacía la cola.
`shuffleQueue`	3554	Mezcla la cola (la actual queda primero).
`setMode`	3562	Cambia el modo de repetición.
`xfDrop`	3574	Libera el <audio> de la canción que se fue.
`xfKill`	3572	Apaga la canción que se iba en la transición (al pausar o cambiar de canción).
`xfCheck`	3576	Decide si empieza la transición del DJ (a 3–5 s del final).
`xfStart`	3585	Cruza la canción que termina con la siguiente usando los dos <audio>.
`updateModeBtn`	3599	Actualiza el botón de repetición.
`loadPeaks`	3601	Carga la forma de onda de una canción.
`cssVar`	3615	Lee una variable CSS del tema (en caché).
`drawWaves`	3607	Dibuja las barras de progreso o la forma de onda.

Función	Línea	Qué hace
roundRect	3631	Rectángulo redondeado en canvas.
updateNow	3641	Actualiza todo lo que muestra la canción actual.
qualPaint	3674	En el reproductor maximizado, cómo llega la canción (p. ej. «Opus · 96 kbps», «Original», «En el teléfono»).
updatePlayState	3664	Íconos de reproducir/pausa y estado de los controles del sistema.
DISC.can	3702	¿Hay puente de Discord (escritorio)?
DISC.update	3702	Programa el aviso a Discord.
DISC.send	3703	Manda a Discord lo que suena (o lo quita).
dcStatusPaint	3719	Línea de estado de Discord en Ajustes.
rcKeep	3740	Audio en silencio para que el sistema mantenga los controles.
sessionHold	3754	Mantiene los controles del sistema mientras carga o al terminar la cola.
rcKeepOff	3760	Detiene el audio en silencio.
stallCheck	3806	Detecta audio atorado a media canción y lo vuelve a pedir.
netBack	3815	Volvió la red: reanuda lo cortado y reconecta.
paintVol	3858	Dibuja la barra de volumen y su ícono.

Letras desde la línea 3863 · 33

Función	Línea	Qué hace
lrcKey	3866	Clave de una letra guardada en el dispositivo.
lrcTime	3867	Formatea un tiempo como mm:ss.cc (LRC).
parseLRC	3868	Lee un archivo LRC (marcas de tiempo y offset).
toLRC	3882	Arma un archivo LRC limpio.
cleanLyricsText	3893	Limpia una letra escrita o pegada (controles, HTML, tamaño).
lyricsFromText	3902	Convierte texto (LRC o simple) en letra.
localLyrics	3903	Letra guardada en este dispositivo.
lyrSame	3911	¿Dos letras son iguales (texto y tiempos)?
lyricsSaved	3919	Muestra la letra recién guardada hasta que Jellyfin la sirva.
getLyrics	3934	Letra de una canción (local, pendiente, caché o servidor).
setLyricsMode	3942	Abre o cierra la letra con transición.
setQueueMode	3955	Abre o cierra la cola del reproductor con transición.
renderSheetQueue	3957	Lista de la cola dentro del reproductor.
screenAwake	3979	Pantalla encendida con la letra a la vista y sonando (Android: keepScreen; si no, Wake Lock).

Función	Línea	Qué hace
applyLyricsMode	3947	Aplica el modo letra sin transición.
reloadLyrics	4003	Vuelve a cargar la letra actual.
renderLyrics	4001	Dibuja la letra de la canción actual.
estimateTimes	4019	Reparte tiempos aproximados para letras sin sincronizar.
syncLyrics	4007	Resalta la línea que suena y desplaza la letra.
refreshLyricsButtons	4071	Botón Letra o Buscar letra según haya letra.
syncStart	4086	Abre el editor para sincronizar a mano.
syncEnd	4100	Cierra el editor.
syncCancel	4104	Cancela la sincronización.
renderSync	4068	Dibuja el editor de sincronización.
syncEditorTick	4112	Marca en el editor la línea que suena.
syncMark	4119	Marca el inicio de la línea resaltada.
syncUndo	4132	Deshace la última marca.
syncNudge	4140	Mueve la última marca ¼ s.
syncSave	4149	Completa los tiempos y guarda el LRC.
saveLyricsText	4163	Guarda una letra en Jellyfin (o en el dispositivo si no se puede).
lyricsSearch	4178	Busca letras en LrcLib y deja elegir una.
lyricsPaste	4208	Diálogo para pegar o editar una letra.
forgetLocalLyrics	4226	Quita la letra guardada en este dispositivo.

Cola desde la línea 4229 · 6

Función	Línea	Qué hace
renderQueue	4231	Dibuja el panel de la cola.
queuePanel	4242	Dibuja el panel lateral de la cola solo si se ve (pantalla ancha o abierto).
renderQueueAll	4243	Redibuja todas las vistas de la cola.
moveQueue	4248	Mueve una canción de la cola (también en el control remoto).
markCurrent	4300	Resalta la canción que suena en las listas.
RC.on	4288	¿Mis dispositivos está activado?

Avisos, menús y diálogos desde la línea 4303 · 2

Función	Línea	Qué hace
toast	4305	Aviso breve en pantalla.
menu	4306	Menú de opciones (hoja inferior).

Mis dispositivos desde la línea 4314 · 62

Función	Línea	Qué hace
devLabel	4319	Nombre amable de un dispositivo.
devName	4321	Nombre de un dispositivo para frases.
expand6	4323	Expande una dirección IPv6.
ipKey	4329	Red y privacidad de una dirección IP.
sameNetwork	4337	¿Dos direcciones están en la misma red?
RC.start	4350	Arranca Mis dispositivos (WebSocket y búsqueda).
RC.stop	4350	Detiene Mis dispositivos.
RC.connect	4350	Abre el WebSocket de Jellyfin con reintentos.
RC.send	4370	Manda un mensaje por el WebSocket.
RC.onMessage	4362	Atiende los mensajes del WebSocket (órdenes, favoritas, keep-alive).
RC.fromMe	4375	¿La orden viene de tu propio usuario?
RC.onPlaystate	4374	Ejecuta pausar, siguiente, buscar… recibidos.
RC.onPlay	4375	Ejecuta una orden de reproducir (y adopta el DJ si viene con él).
RC.onGeneral	4424	Ejecuta repetir o mezclar recibidos.
RC.queueForReport	4431	Cola que se informa a Jellyfin.
RC.reportSoon	4400	Informa el estado a Jellyfin en un momento.
RC.schedule	4438	Programa la siguiente búsqueda de dispositivos.
RC.poll	4438	Busca tus otros Nanuq y sigue al que controlas.
RC.maybeFollow	4459	Se conecta solo a otro Nanuq que suena en tu red.
RC.enter	4465	Empieza a controlar otro dispositivo.
RC.leave	4456	Deja de controlar y recupera tu cola.
RC.apply	4458	Refleja aquí lo que suena en el otro dispositivo.
RC.pos	4477	Segundo estimado de la canción en el otro dispositivo.
RC.tick	4520	Actualiza barra, letra y controles del sistema en control remoto.
RC.decorate	4531	Indica «Sonando en …» en el reproductor.
RC.fail	4537	Avisa que una orden no se pudo enviar.
RC.cmd	4538	Manda una orden al dispositivo que suena.
RC.seek	4544	Salta en el otro dispositivo.
RC.window	4545	Ventana de la cola que se manda (≤ 200).
RC.playQueue	4546	Manda una cola para reproducir allá.
RC.add	4555	Agrega canciones a la cola de allá.

Función	Línea	Qué hace
RC.reorder	4565	Manda la cola reordenada (conservando el DJ).
RC.general	4576	Manda una orden general allá.
RC.here	4578	«Reproducir aquí»: trae la música (y el DJ) a este dispositivo.
RC.there	4601	Pasa la música (y el DJ) a otro dispositivo.
__nanuqTick	4620	Latido cada 20 s desde Android (WebSocket, fiesta, audio atorado).
__nanuqVis	4628	Android avisa si la app está a la vista.
openDevices	4629	Menú Mis dispositivos.
updateFavBtn	4660	Botón de favorita del reproductor.
setFavLocal	4666	Marca la favorita en la cola local.
favPending	4673	Recuerda un cambio de favorita reciente.
favMerge	4674	Aplica los cambios recientes sobre la lista del servidor.
favHold	4692	Protege la favorita de la canción que suena (fallo de Jellyfin 12).
favHeld	4695	¿Se está protegiendo esta favorita?
favReassert	4696	Vuelve a guardar la favorita después de cada aviso de reproducción.
favRemote	4704	Atiende cambios de favoritas hechos en otro lado.
refreshFav	4726	Confirma con el servidor si la canción es favorita.
toggleFav	4737	Marca o quita la favorita y lo comprueba.
favConfirm	4751	Segunda confirmación contra la sección de favoritas.
updateDlBtn	4771	Botón Descargar del reproductor.
dlAddToast	4780	Aviso al pedir una descarga.
dlTap	4789	Toque en Descargar (pedir o administrar).
connTest	4802	«Probar conexión»: mide API, carátula, audio, saltos y reproductor.
startRadio	4930	Radio a partir de una canción.
addToPlaylistMenu	4944	Menú para agregar a una playlist (o crear una).
songOptions	4959	Opciones comunes de una canción.
queueMenu	4966	Menú de una canción de la cola.
trackMenu	4979	Menú de una canción de una lista.
askConfirm	4987	Diálogo de confirmación.
askName	4993	Diálogo para escribir un nombre.
openSheet	5001	Abre el reproductor maximizado.
closeSheet	5009	Cierra el reproductor maximizado.

Fiesta (watch party) desde la línea 5012 · 65

Función	Línea	Qué hace
partyHost	5019	¿Este dispositivo puede ser anfitrión (escritorio)?
partyGuest	5020	¿Este dispositivo es invitado (Android)?
partyOn	5021	¿Hay fiesta en este dispositivo?
partySongUrl	5023	URL para mandar una canción a la fiesta (streaming a 320 kbps, convertida a MP3 si hace falta).
partyCover	5026	Miniatura JPEG de 240 px de la carátula.
partyCoverOk	5036	¿La carátula viene de la sala (127.0.0.1 con token)?
partyClean	5039	Limpia el estado que manda la computadora (no confiable).
stopForParty	5051	Detiene tu música al abrir la sala.
PH.active	5066	¿La fiesta maneja los controles?
PH.init	5067	Prepara el reproductor de la fiesta y escucha a main.js.
PH.onEv	5076	Atiende los eventos de la sala (estado, llegó, salió).
PH.apply	5077	Aplica el estado de la sala (modo fiesta de la computadora).
PH.list	5097	Lista de canciones de la sala.
PH.open	5062	Abre la sala.
PH.close	5104	Cierra la sala.
PH.jump	5106	Pone ya una canción de «A continuación».
PH.order	5112	Cambia el orden de la lista.
PH.next	5095	Sigue la lista: pone la siguiente que ya llegó.
PH.play	5119	Reproduce una canción de la fiesta.
PH.stopCur	5093	Detiene la canción actual de la fiesta.
PH.done	5137	Terminó una canción: la borra y pasa a la siguiente.
PH.toggle	5138	Pausa o reanuda la fiesta.
PH.yieldTo	5151	Pusiste tu música: la fiesta se pausa y cede los controles.
PH.media	5127	Controles del sistema con la canción de la fiesta.
PH.report	5170	Avisa a la sala qué suena.
PH.loadLyrics	5175	Carga la letra de la canción de la fiesta.
PH.line	5182	Línea de letra actual y la siguiente.
PH.tick	5173	Karaoke: cada segundo, tiempo y línea a los teléfonos.
PG.connect	5203	Entra a una sala (con código o llave) y reconecta solo.
PG.leave	5204	Sale de la fiesta.
PG.joined	5240	¿Estás en una fiesta?
PG.ui	5207	Modo fiesta del teléfono (oculta barras y reproductor).

Función	Línea	Qué hace
`PG.wake`	5250	El teléfono despertó: vuelve a entrar si hace falta.
`PG.resume`	5256	Al abrir la app, vuelve a la fiesta en curso.
`PG.abortUp`	5216	Cancela la subida en curso.
`PG.onMsg`	5262	Atiende los mensajes de la sala.
`PG.add`	5289	Manda una canción a la fiesta (letra, carátula y audio por partes).
`PG.remove`	5345	Cancela una canción que todavía llega.
`PG.find`	5347	Busca salas en la red (UDP desde Android).
`partyAddr`	5367	Convierte la dirección escrita en ws://…/fiesta.
`iglooHTML`	5378	HTML del iglú.
`iglooBear`	5394	Nanuq de un invitado con el disfraz de su tema.
`iglooHoldSVG`	5401	Carátula en alto con destellos (quien escogió la canción).
`iglooHeldOnTop`	5412	Pone encima de la carátula el objeto que traía el Nanuq.
`partyGuestColor`	5423	Color de la etiqueta según el tema del invitado.
`partyCoverSrc`	5431	URL de la carátula de una canción de la fiesta.
`iglooSync`	5439	Coloca a los invitados en el círculo sin redibujar.
`iglooLive`	5471	Anima el iglú solo mientras se ve.
`iglooQueue`	5478	Fila «A continuación» a pantalla completa.
`iglooQueueEdge`	5487	Degradado si hay más canciones fuera de la vista.
`partyVolumeStep`	5566	Volumen con ↑ ↓ a pantalla completa y su aviso.
`iglooIdle`	5579	Esconde el puntero tras 2.5 s quieto.
`iglooVinyl`	5589	Funda y disco de vinilo de la canción que suena.
`iglooLine`	5601	Letra en el techo con destello.
`renderPartySoon`	5608	Redibuja la fiesta en un momento.
`partyPaintTick`	5612	Actualiza el karaoke (línea, siguiente, tiempo).
`partyPaintUp`	5618	Progreso de la canción que mandas.
`partyRowsHTML`	5619	Lista de canciones de la fiesta.
`partyGuestsHTML`	5628	Lista de invitados.
`partyNowHTML`	5629	Lo que suena en la fiesta.
`paintParty`	5610	Dibuja la pantalla de la fiesta (anfitrión o invitado).
`partySongsHTML`	5695	Canciones para mandar.
`partySearch`	5696	Buscador de la fiesta (playlists, artistas, álbumes, canciones, letras).
`partyBrowse`	5724	Canciones de un artista, álbum o playlist en la fiesta.

Función	Línea	Qué hace
V.party	5739	Pantalla Fiesta.

Acciones globales desde la línea 5753 · 1

Función	Línea	Qué hace
d0	5755	Atajo del dataset de un elemento.

Entrar desde la línea 5914 · 11

Función	Línea	Qué hace
recentServers	5917	Servidores usados antes.
rememberServer	5918	Recuerda un servidor.
forgetServer	5924	Olvida un servidor.
probeTimed	5926	Prueba un servidor midiendo cuánto tarda.
canDiscover	5937	¿Se pueden buscar servidores en la red?
discoverServers	5939	Busca Jellyfin en la red (UDP 7359).
serverList	5958	Lista de servidores con su estado.
cleartextRisk	5985	¿La contraseña viajaría sin cifrar por internet?
showLogin	5974	Pantalla de entrar.
startApp	6056	Arranca la app con la sesión (cola, posición, DJ, Inicio).
logout	6077	Cierra sesión y borra lo personal.

17 Resto de la página

Idiomas y service worker.



web/i18n.js

1040 líneas · 1 funciones

Función	Línea	Qué hace
tx	1031	Traduce un texto en español al idioma elegido y pone los valores {0}, {1}...

web/sw.js

45 líneas · 3 funciones

Función	Línea	Qué hace
install	8	Guarda en caché los archivos de la app al instalar el service worker.
activate	12	Borra las cachés de versiones anteriores.
fetch	16	Sirve solo los archivos de la app (red primero para HTML/JS/CSS); la API y la música van directo a la red.

18 Android

Java nativo y el puente de JavaScript.



android/app/src/main/java/com/nanuq/app/Discovery.java

78 líneas · 3 funciones

Función	Línea	Qué hace
`Discovery.Discovery`	27	Constructor privado (solo métodos estáticos).
`Discovery.find`	30	Busca por UDP (Jellyfin o fiestas) y devuelve las respuestas en JSON.
`Discovery.send`	72	Manda la pregunta a todas las direcciones de difusión.

android/app/src/main/java/com/nanuq/app/Downloads.java

595 líneas · 44 funciones

Función	Línea	Qué hace
`Downloads.get`	43	Instancia única de las descargas.
`Downloads.Downloads`	72	Prepara carpeta, índice, servidor local y vigilancia de red.
`Downloads.audio`	83	Archivo de audio para el servidor local.
`Downloads.audioMime`	84	Tipo del audio para el servidor local.
`Downloads.cover`	85	Carátula para el servidor local.
`Downloads.prefetch`	86	Canción guardada por adelantado para el servidor local.
`Downloads.prefetchMime`	87	Tipo de la canción guardada por adelantado.
`Downloads.onAvailable`	91	Volvió la red: reintenta las descargas.
`Downloads.onCapabilitiesChanged`	92	Cambió la red: revisa la cola de descargas.
`Downloads.setListener`	97	Conecta la página para recibir cambios.
`Downloads.configure`	101	Máximo de canciones y solo Wi-Fi.
`Downloads.base`	117	Dirección del servidor local.
`Downloads.okUrl`	127	¿La URL tiene la forma de audio o carátula de Jellyfin?
`Downloads.openSameHost`	141	Abre una conexión siguiendo redirecciones solo al mismo servidor.
`Downloads.enqueue`	162	Agrega una descarga (o la actualiza).
`Downloads.setScore`	196	Actualiza cuántas veces sonó una descarga.
`Downloads.remove`	202	Quita una descarga.
`Downloads.removeAll`	208	Quita todas (o solo las automáticas).
`Downloads.list`	214	Lista de descargas en JSON.

Función	Línea	Qué hace
`Downloads.network`	222	Red actual (wifi, datos, ninguna), mirando debajo de una VPN.
`Downloads.underlying`	236	Red real debajo de una VPN.
`Downloads.file`	245	Archivo de una descarga lista.
`Downloads.coverFile`	253	Carátula de una descarga.
`Downloads.setCover`	262	Pide la carátula de una descarga vieja.
`Downloads.fetchCoverLater`	270	Baja la carátula en segundo plano.
`Downloads.fetchCover`	273	Baja la carátula (≤ 15 MB).
`Downloads.mime`	301	Tipo de audio de una descarga.
`Downloads.retryAndKick`	309	Reintenta las fallidas y sigue.
`Downloads.kick`	316	Arranca la cola de descargas.
`Downloads.kickLocked`	318	Arranca la cola (ya con el candado).
`Downloads.loop`	324	Baja las pendientes de una en una.
`Downloads.nextPending`	344	La siguiente (primero las pedidas por ti).
`Downloads.download`	354	Baja una canción con topes y espacio libre; Opus solo si está completo (OggCheck).
`Downloads.victim`	431	La automática menos escuchada (para hacer lugar).
`Downloads.removeEntry`	440	Borra una descarga y sus archivos.
`Downloads.caps`	454	Capacidades de la red activa.
`Downloads.unmetered`	459	¿La red es sin medición (Wi-Fi, Ethernet)?
`Downloads.networkAllowed`	464	¿Se puede descargar con esta red?
`Downloads.safe`	472	Nombre de archivo seguro para un id.
`Downloads.toJson`	474	Descarga en JSON para la página.
`Downloads.emit`	492	Avisa a la página un cambio.
`Downloads.ensureLoaded`	498	Lee el índice cuando el almacén se abre.
`Downloads.load`	500	Lee el índice cifrado (o recupera desde los archivos).
`Downloads.save`	566	Guarda el índice cifrado.

`android/app/src/main/java/com/nanuq/app/LocalServer.java` 184 líneas · 7 funciones

Función	Línea	Qué hace
`LocalServer.LocalServer`	46	Abre el servidor en 127.0.0.1 con token aleatorio.
`LocalServer.base`	60	Dirección base con el token.
`LocalServer.acceptLoop`	62	Acepta conexiones (máx. 32 a la vez).
`LocalServer.readLine`	74	Lee una línea de la petición con tope y hora límite.
`LocalServer.handle`	86	Atiende una petición (10 s y ≤ 100 encabezados).

Función	Línea	Qué hace
`LocalServer.status`	129	Responde un código de estado.
`LocalServer.serveFile`	135	Entrega un archivo con rangos (saltar dentro de la canción).

android/app/src/main/java/com/nanuq/app/Page.java

458 líneas · 45 funciones

Función	Línea	Qué hace
`Page.run`	83	Latido cada 20 s hacia la página.
`Page.create`	145	Crea y configura el WebView (sin acceso a archivos, puente) y conecta música, controles y descargas.
`Page.recreate`	188	Rehace el WebView si el sistema lo cerró.
`Page.dispatchMedia`	198	Pasa un botón de la notificación a la página.
`Page.showNotice`	217	Notificación de «canción solicitada lista».
`Page.keepScreen`	250	Puente: la página pide la pantalla encendida (se recuerda si no hay pantalla).
`Page.setMetadata`	245	Puente: título, artista, álbum y carátula.
`Page.setPlaybackState`	246	Puente: reproduciendo o en pausa.
`Page.setActions`	247	Puente: botones disponibles.
`Page.setPosition`	248	Puente: posición y duración.
`Page.dlList`	269	Puente: lista de descargas.
`Page.dlEnqueue`	270	Puente: descargar una canción.
`Page.dlCover`	271	Puente: pedir una carátula.
`Page.dlBase`	272	Puente: dirección del servidor local.
`Page.dlRemove`	273	Puente: quitar una descarga.
`Page.dlRemoveAll`	274	Puente: quitar descargas.
`Page.dlConfig`	275	Puente: ajustes de descargas.
`Page.dlScore`	276	Puente: reproducciones de una descarga.
`Page.dlNetwork`	277	Puente: tipo de red.
`Page.pfSet`	280	Puente: ventana de lo que sigue por adelantado.
`Page.pfReady`	281	Puente: ¿ya está guardada esa canción?
`Page.pfBroken`	282	Puente: esa canción guardada no sonó.
`Page.http`	285	Puente: petición a SomeDL sin CORS.
`Page.secKind`	293	Puente: estado del almacén cifrado.
`Page.secGet`	294	Puente: leer un dato cifrado.
`Page.secSet`	295	Puente: guardar un dato cifrado.
`Page.secDel`	296	Puente: borrar un dato cifrado.
`Page.secReset`	298	Puente: empezar de cero.

Función	Línea	Qué hace
`Page.videoBegin`	302	Puente: empieza a guardar un video (MP4/WebM).
`Page.videoChunk`	314	Puente: un trozo del video (≤ 300 MB).
`Page.videoEnd`	326	Puente: termina y guarda en Películas › Nanuq.
`Page.videoShare`	368	Puente: compartir el video recién guardado.
`Page.notice`	377	Puente: notificación del sistema.
`Page.partyDiscover`	380	Puente: buscar fiestas por UDP.
`Page.discover`	388	Puente: buscar Jellyfin por UDP.
`Page.onRenderProcessGone`	406	El proceso del WebView murió: se rehace.
`Page.onPageFinished`	400	Sin pantalla: le dice a la página que no está a la vista.
`Page.shouldInterceptRequest`	412	Sirve la app desde los assets en http://localhost.
`Page.shouldOverrideUrlLoading`	429	Solo enlaces http(s) salen al navegador.
`Page.mime`	302	Tipo de un archivo de los assets.
`Page.attach`	99	La pantalla adopta la página (la que arrancó sin pantalla o una nueva).
`Page.detach`	117	La pantalla se cierra: la página se va con ella, salvo que siga sonando.
`Page.resumeHeadless`	131	▶ con Nanuq cerrado: arranca la página sin pantalla y sigue la cola.
`Page.played`	142	Ya sonó tras retomar sin pantalla: la página ya no se apaga sola a los 30 s.
`Page.startOutside`	210	Abre algo fuera de la app (navegador, compartir), también sin pantalla.

`android/app/src/main/java/com/nanuq/app/MainActivity.java` 85 líneas · 7 funciones

Función	Línea	Qué hace
`MainActivity.onCreate`	24	Muestra la página (la que ya existía sin pantalla o una nueva) y pide permiso de notificaciones.
`MainActivity.onBackPressed`	39	Atrás: la página decide o la app va al fondo.
`MainActivity.onStart`	48	La app volvió a la vista.
`MainActivity.onStop`	51	La app pasó al fondo (guarda el almacén).
`MainActivity.onDestroy`	54	Suelta la página: se va con la pantalla salvo que el sistema la cierre mientras suena.
`MainActivity.setKeepOn`	60	Enciende o suelta FLAG_KEEP_SCREEN_ON de la ventana.
`MainActivity.setBarColor`	65	Color de las barras del sistema según el tema.

android/app/src/main/java/com/nanuq/app/MediaCenter.java 399 líneas · 34 funciones

Función	Línea	Qué hace
MediaCenter.get	50	Instancia única del centro multimedia.
MediaCenter.MediaCenter	70	Crea la MediaSession y el canal de notificación.
MediaCenter.onPlay	81	Botón reproducir del sistema.
MediaCenter.onPause	82	Botón pausa del sistema.
MediaCenter.onStop	83	Botón detener del sistema.
MediaCenter.onSkipToNext	84	Botón siguiente del sistema.
MediaCenter.onSkipToPrevious	85	Botón anterior del sistema.
MediaCenter.onSeekTo	86	Saltar desde el sistema.
MediaCenter.onFastForward	87	Adelantar desde el sistema.
MediaCenter.onRewind	88	Retroceder desde el sistema.
MediaCenter.attach	99	Conecta la página.
MediaCenter.detach	101	Desconecta la página.
MediaCenter.hasPage	104	¿Hay página que reciba órdenes?
MediaCenter.stopIfNoPage	107	Sin página, quita la notificación.
MediaCenter.dispatch	116	Manda una acción a la página.
MediaCenter.setMetadata	144	Datos de la canción (y carga la carátula).
MediaCenter.setPlaybackState	168	Estado de reproducción.
MediaCenter.setActions	180	Botones disponibles.
MediaCenter.setPosition	188	Posición y duración.
MediaCenter.isPlaying	202	¿Está sonando?
MediaCenter.currentPosition	204	Posición actual estimada.
MediaCenter.publishMetadata	209	Publica los datos en la MediaSession.
MediaCenter.publishState	219	Publica el estado y los botones.
MediaCenter.refresh	236	Ajusta servicio, notificación y bloqueos.
MediaCenter.shutdown	266	Detiene todo al cerrar la app.
MediaCenter.buildNotification	277	Arma la notificación de reproducción.
MediaCenter.action	43	Un botón de la notificación.
MediaCenter.serviceIntent	311	Intent de un botón hacia el servicio.
MediaCenter.contentIntent	319	Abrir la app desde la notificación.
MediaCenter.onReceive	327	Audífonos desconectados: pausa.
MediaCenter.setNoisyReceiver	332	Escucha la desconexión de audífonos.
MediaCenter.loadBitmap	346	Carga la carátula con topes (8 MB, reducida).
MediaCenter.readCapped	391	Lee con tope de bytes.

Función	Línea	Qué hace
`MediaCenter.holdForResume`	128	Al reanudar con Nanuq cerrado: el servicio pasa a primer plano ya, con la última canción en pausa.

android/app/src/main/java/com/nanuq/app/MediaButtonReceiver.java

31 líneas · 1 funciones

Función	Línea	Qué hace
`MediaButtonReceiver.onReceive`	18	Con Nanuq cerrada, ▶ de los audífonos o del sistema la despierta y sigue la cola.

android/app/src/main/java/com/nanuq/app/Resume.java

74 líneas · 2 funciones

Función	Línea	Qué hace
`Resume.save`	28	Guarda la última canción (cifrada) para la tarjeta de reanudar.
`Resume.clear`	35	Borra la última canción (al cerrar sesión).

android/app/src/main/java/com/nanuq/app/NanuqBrowserService.java

72 líneas · 2 funciones

Función	Línea	Qué hace
`NanuqBrowserService.onGetRoot`	37	Responde a la interfaz de Android (solo apps del sistema) si hay algo que reanudar.
`NanuqBrowserService.onLoadChildren`	43	La última canción, como elemento que se puede reproducir.

android/app/src/main/java/com/nanuq/app/NativeHttp.java

84 líneas · 5 funciones

Función	Línea	Qué hace
`NativeHttp.NativeHttp`	19	Constructor privado (solo métodos estáticos).
`NativeHttp.Result`	24	Respuesta (estado y texto).
`NativeHttp.allowed`	30	¿Ruta permitida de SomeDL?
`NativeHttp.fetch`	37	Petición sin CORS solo a SomeDL, sin redirecciones.
`NativeHttp.readAll`	71	Lee la respuesta (≤ 4 MB).

android/app/src/main/java/com/nanuq/app/PlaybackService.java

108 líneas · 7 funciones

Función	Línea	Qué hace
`PlaybackService.onCreate`	32	Prepara los bloqueos de CPU y Wi-Fi.
`PlaybackService.onStartCommand`	47	Primer plano y botones de la notificación.
`PlaybackService.setLocks`	68	Toma o suelta los bloqueos.
`PlaybackService.release`	83	Sale del primer plano y termina.
`PlaybackService.onTaskRemoved`	91	Quitada de recientes: cierra todo.
`PlaybackService.onDestroy`	99	Suelta los bloqueos.
`PlaybackService.onBind`	106	Sin enlace (servicio iniciado).

android/app/src/main/java/com/nanuq/app/Prefetch.java 222 líneas · 16 funciones

Función	Línea	Qué hace
Prefetch.get	40	Instancia única de la caché por adelantado.
Prefetch.Prefetch	61	Vacía la caché anterior y vigila la red.
Prefetch.onAvailable	71	Volvió la red: sigue bajando.
Prefetch.set	80	Recibe la ventana deseada y borra lo que sobra.
Prefetch.broken	109	Una guardada no sonó: se borra y no se repite.
Prefetch.ready	116	¿Está lista esa canción?
Prefetch.file	118	Archivo de una canción lista.
Prefetch.mime	125	Tipo de audio.
Prefetch.drop	130	Quita una canción y su archivo.
Prefetch.deleteFiles	132	Borra los archivos de una canción.
Prefetch.kick	139	Arranca las descargas.
Prefetch.kickLocked	141	Arranca (ya con el candado).
Prefetch.loop	147	Baja las pendientes de una en una (respeta el tope total).
Prefetch.online	160	¿Hay red?
Prefetch.fetch	166	Baja una canción con topes; Opus solo si está completo (OggCheck).
Prefetch.trim	211	Si se pasa del tope total, suelta las más lejanas.

android/app/src/main/java/com/nanuq/app/SecureStore.java 234 líneas · 16 funciones

Función	Línea	Qué hace
SecureStore.get	44	Instancia única (o lee un valor si recibe una clave).
SecureStore.SecureStore	66	Abre el almacén al crearse.
SecureStore.open	77	Abre clave (Keystore) y archivo: listo, locked o lost.
SecureStore.kind	52	Estado (reintenta si estaba locked).
SecureStore.reset	125	Empezar de cero: aparta el archivo y crea clave nueva.
SecureStore.ready	138	¿Está abierto?
SecureStore.set	154	Guarda un valor (≤ 8 MB).
SecureStore.del	162	Borra un valor.
SecureStore.validKey	169	¿Clave válida?
SecureStore.scheduleSave	171	Guarda en 400 ms.
SecureStore.flush	177	Guarda ya.
SecureStore.saveNow	179	Cifra y escribe el archivo.
SecureStore.encrypt	203	Cifra con AES-256-GCM.
SecureStore.decrypt	213	Descifra y verifica.

Función	Línea	Qué hace
`SecureStore.readAll`	220	Lee un archivo (≤ 64 MB).
`SecureStore.writeAtomic`	228	Escribe de forma atómica.

android/bridge/android-bridge.js

255 líneas · 12 funciones

Función	Línea	Qué hace
`abs`	17	Convierte una URL relativa en absoluta.
`MediaMetadata`	19	Reemplazo de MediaMetadata para la página.
`bestArt`	28	Elige la carátula más adecuada (≤ 512 px) para la notificación.
`mediaSession.metadata`	42	Manda título, artista, álbum y carátula a Android.
`mediaSession.playbackState`	47	Manda el estado de reproducción a Android.
`mediaSession.setActionHandler`	54	Registra los botones que maneja la página.
`mediaSession.setPositionState`	58	Manda la posición solo cuando se desvía.
`__nanuqMedia`	234	Ejecuta en la página un botón de la notificación o de los audífonos.
`sendTheme`	245	Manda el color del tema a las barras del sistema.
`NanuqNativeAudio`	220	Objeto que se usa como un <audio> y reproduce con el reproductor del sistema (o un <audio> por dentro en el modo demo).
`NanuqNativeAudio._ev`	183	Convierte cada aviso del teléfono en los eventos de un <audio>.
`__nanuqAudio`	221	Recibe los avisos de NativeAudio; ignora los de una canción anterior.

android/build-apk.sh

27 líneas · 1 funciones

Función	Línea	Qué hace
`(script)`	1	Compila el APK y lo deja en dist/<versión>/.

android/app/src/main/java/com/nanuq/app/OggCheck.java

81 líneas · 2 funciones

Función	Línea	Qué hace
`OggCheck.valid`	47	Revisa que un Ogg (Opus) tenga todas sus páginas completas y con CRC correcto; el fin del flujo (EOS) solo si el servidor no dijo el tamaño.
`OggCheck.mime`	75	Acepta el tipo de audio del servidor solo con la forma audio/....

android/app/src/main/java/com/nanuq/app/NativeAudio.java

432 líneas · 10 funciones

Función	Línea	Qué hace
`NativeAudio.doSrc`	131	Prepara una canción en un MediaPlayer (solo http/https) con sus avisos.

Función	Línea	Qué hace
`NativeAudio.doPlay`	183	Pide el foco de audio y empieza a sonar.
`NativeAudio.onAudioFocusChange`	353	Qué hacer cuando otra app toma o devuelve el audio.
`NativeAudio.pauseForLoss`	373	Baja con desvanecido y pausa lo que sonaba al perder el foco.
`NativeAudio.resumeLost`	396	Vuelve a sonar lo pausado por el foco, subiendo con desvanecido.
`NativeAudio.resumeCheck`	408	Tras perder el foco: si ya no suena nada en el teléfono (≈ 4 s), vuelve a sonar (hasta 5 min).
`NativeAudio.releaseAll`	115	Para y suelta todos los reproductores (la página se fue o se recargó).
`NativeAudio.stopSoft`	235	Suelta la canción que suena bajándola a 0 en 80 ms (cortarla de golpe chasqueaba).
`NativeAudio.deClick`	248	Sube (60 ms al empezar) o baja (antes de pausar) el volumen de una canción sin chasquidos.
`NativeAudio.ramp`	259	Rampa de volumen en el hilo principal (la usan el foco, el anti-chasquido y el soltar suave).

19 Escritorio

Electron y su empaquetado.



linux/main.js

866 líneas · 63 funciones

Función	Línea	Qué hace
serve	47	Servidor interno 127.0.0.1:47863 que entrega la app a la ventana (y la fiesta), endurecido.
broadcastOf	79	Dirección de difusión de una red IPv4.
discover	80	Busca servidores Jellyfin por UDP 7359.
fromApp	100	¿El mensaje IPC viene de la página de la app?
vaultBackend	132	Qué llavero del sistema hay (GNOME Keyring, KWallet, DPAPI, Llavero).
secFile	148	Ruta del archivo de datos cifrado.
secKeyFile	149	Ruta de la clave de datos cifrada con el llavero.
writeAtomic	157	Escribe un archivo de forma atómica (0600).
secInit	160	Abre el almacén cifrado (AES-256-GCM): locked, lost o listo.
secSeal	155	Cifra un texto con AES-256-GCM (iv, etiqueta y datos).
secOpen	156	Descifra y comprueba un archivo del almacén (falla si se alteró).
secFlush	196	Escribe el almacén cifrado en el disco.
dcSet	239	Anota el estado de la conexión con Discord.
dcPaths	240	Sockets o tuberías de Discord posibles (solo los tuyos).
dcWrite	251	Manda un mensaje con el formato de Discord.
dcRetry	252	Vuelve a buscar Discord en 10 s.
dcConnect	253	Se conecta a Discord probando cada socket y saluda.
dcData	269	Lee los mensajes de Discord con topes (READY, ERROR, cierre, ping).
dcFlush	280	Manda la actividad respetando el ritmo de Discord (y la mínima si la rechazó).
dcActivity	303	Arma la actividad «Escuchando» limpia y acotada.
jparse	340	Lee JSON de afuera sin toString/valueOf/__proto__.
ptStr	341	Convierte a texto seguro y acotado (sin controles).
ptIp	342	Dirección IP sin el prefijo ::ffff:.

Función	Línea	Qué hace
ptLocal	346	¿La dirección es de tu red local?
ptOpenNow	353	¿La sala está abierta?
ptEmit	354	Avisa a la página un evento de la sala.
ptBase	359	Carpeta en disco donde se guardan las canciones de la sala (en Linux, ~/.cache/nanuq; antes era la temporal, que en el Flatpak es RAM).
ptCleanTemp	364	Borra carpetas temporales de salas anteriores.
ptNonce	377	Nonce de AES-GCM de un bloque: 8 bytes al azar de la canción + el número de bloque.
ptSeal	378	Cifra y autentica un bloque de 64 KB de una canción de la fiesta con la clave de la sala.
ptEncSize	383	Tamaño exacto que debe tener el archivo cifrado (detecta archivos cortados o alargados).
ptOpenRange	384	Lee, comprueba y descifra solo los bloques de un rango pedido; falla si un bloque fue alterado.
ptSniff	400	¿Es audio? (por sus primeros bytes).
ptCover	413	Valida la carátula que manda un teléfono (JPEG/PNG/WebP ≤ 150 KB).
ptPublic	422	Datos públicos de una canción de la lista.
ptGuests	426	Invitados presentes y ausentes.
ptAddrs	427	Direcciones IPv4 de esta computadora.
ptStatus	432	Estado completo de la sala para la página.
ptState	438	Avisa el estado a todos (agrupado en 150 ms).
ptStateNow	439	Manda el estado ya.
ptKill	445	Quita una canción de la lista y del disco.
ptFrame	454	Escribe un marco WebSocket.
ptSend	450	Manda un mensaje JSON a un teléfono.
ptDrop	462	Cierra la conexión de un teléfono (con motivo).
ptGone	468	Un teléfono se desconectó (queda ausente o salió).
ptUpgrade	481	Acepta la conexión WebSocket (solo red local, topes).
ptData	496	Lee marcos WebSocket con todas las validaciones.
ptMsg	525	Atiende un mensaje de un teléfono.
ptNewCode	547	Cambia el código de la sala.
ptHello	539	Entrada de un teléfono: código, llave, fuerza bruta.
ptAdd	581	Un teléfono anuncia una canción (con topes).
ptChunk	599	Recibe un pedazo de la canción y lo escribe al disco.

Función	Línea	Qué hace
ptEnd	617	La canción terminó de llegar: comprueba que sea audio.
ptServe	634	Sirve canciones y carátulas de la sala a la ventana (con Range).
ptGuestCover	662	Sirve carátulas a los teléfonos (solo con su llave).
ptGc	671	Limpieza cada 30 s (viejas, a medias, ausentes, mudas).
ptListen	681	Abre el puerto de la sala (47864 o uno libre).
ptOpen	694	Abre la sala: servidor, UDP, código y token.
ptClose	721	Cierra la sala y corta todas las conexiones.
stateFile	781	Archivo con el tamaño y posición de la ventana.
loadState	783	Lee el tamaño y posición de la ventana.
saveState	788	Guarda el tamaño y posición de la ventana.
start	794	Arranca la app: almacén, servidor, ventana, permisos y protecciones.

linux/preload.js

25 líneas · 6 funciones

Función	Línea	Qué hace
nanuqDesktop.discover	7	Puente: buscar Jellyfin en la red.
nanuqDesktop.http	8	Puente: petición sin CORS a SomeDL.
nanuqDesktop.discord	11	Puente: actividad de Discord (o su estado).
nanuqDesktop.party.op	14	Puente: operaciones de la fiesta (abrir, cerrar, saltar, ordenar, karaoke…).
nanuqDesktop.party.on	15	Puente: escuchar los eventos de la sala.
nanuqDesktop.secure.*	17	Puente síncrono al almacén cifrado (kind, get, set, del, reset).

linux/prepare-web.js

16 líneas · 1 funciones

Función	Línea	Qué hace
(script)	3	Copia web/ a www/ y toma la versión de app.js.

linux/collect.js

15 líneas · 1 funciones

Función	Línea	Qué hace
(script)	3	Mueve los paquetes compilados a dist/<versión>/.

linux/flatpak/build-flatpak.sh

54 líneas · 1 funciones

Función	Línea	Qué hace
(script)	1	Arma el Flatpak, lo publica en el repositorio local firmado y lo instala o actualiza.

20 Compilación

El script que compila todo.



scripts/build.sh

73 líneas · 1 funciones

Función	Línea	Qué hace
(script)	1	Sube la versión y compila Android, Linux, Flatpak y Windows.

Hecho con cariño (y mucho hielo)

Si cambias algo, recuerda: la interfaz vive en web/ y cada texto pasa por tx().

